

C#从入门到精通

2026年3月18日星期三 上午9:26

本课程需要有一定的C语言基础

C#语言入门详解

Learning C# Programming Language in Detail

<http://www.wasbec.org>

刘铁猛《C#语言入门详解》全集

从 15:01 开始分享

获取视频分享链接

手机扫码观看/分享

动态

微信

QQ

QQ空间

微博

贴吧

嵌入代码

一、Hello World程序

```
文件(F) 编辑(E) 视图(V) Git(G) 项目(P) 生成(B) 调试(D) 测试(S) 工具(T) 扩展(X) 窗口(W) 帮助(H) 搜索  C#test
Debug  Any CPU  C#test  Main(string[] args)
Program.cs*
1  using System; 使用System命名空间
2  namespace C_test
3  {
4      0 个引用
5      internal class Program Program类为主类
6      {
7          0 个引用
8          static void Main(string[] args)
9          {
10             System.Console.WriteLine("Hello, World!");
11             调用System命名空间的Console类的WriteLine方法
12         }
13     }
14 }
```

可以发现一个hello world程序在一个类class里面和一个命名空间里面namespace . 由此可以引申出类和对象的定义。

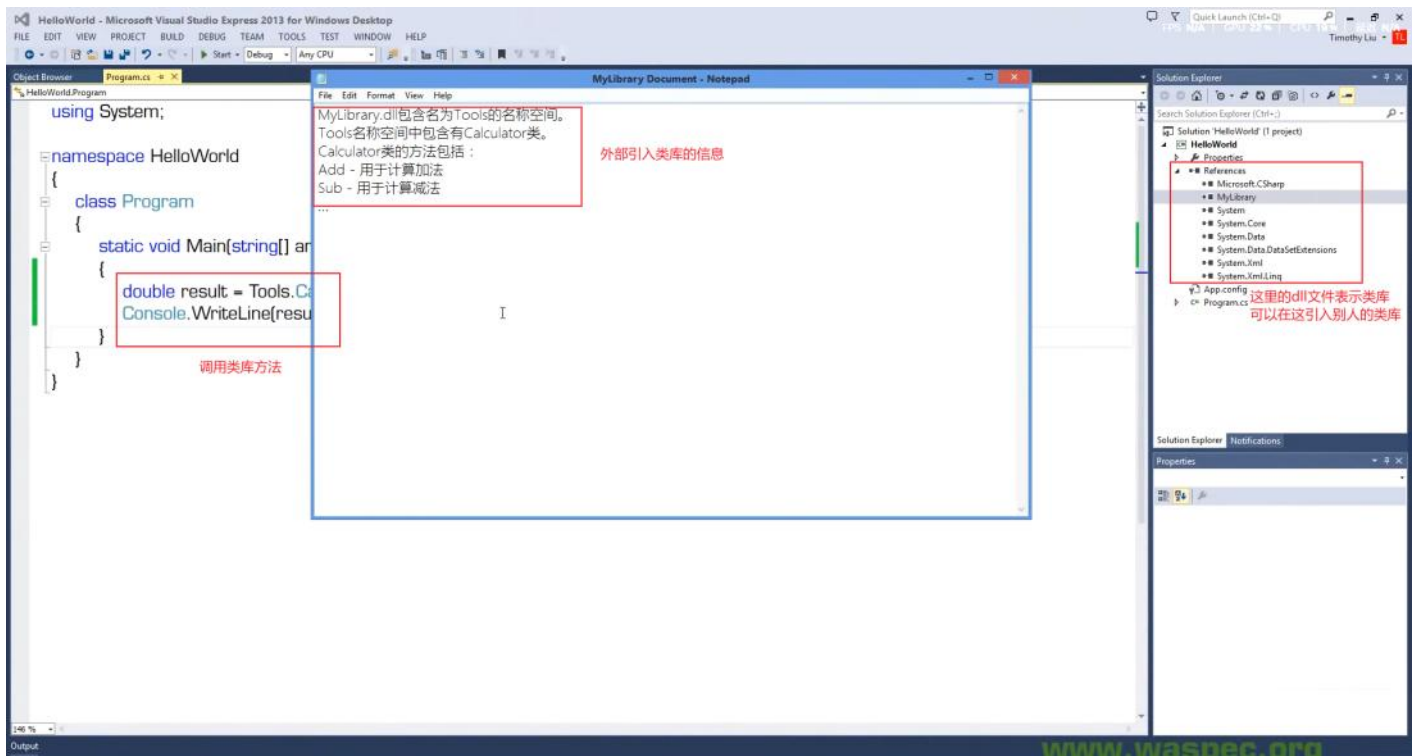
1、类和命名空间

类class是构成程序的主体

命名空间namespace是以树形结构组织类

注意相同的类名，功能不一定相同，因为他们可能在不同的命名空间内。不同命名空间里的同一类名表示不同的类。

2、类库



二、类与对象的关系

类就是现实世界的事物的模型，抽象的过程相当于建模，一个类具有自身对应的属性，也有对应的方法。



类与对象的关系

- 对象也叫实例，是类经过“实例化”后得到的内存中的实体
 - Formally “instance” is synonymous with “object”——对象和实例是一回事
 - “飞机”与“一架飞机”有何区别？天上有（一架）飞机——必需是实例飞，概念是不能飞的
 - 有些类是不能实例化的，比如“数学”（ Math class ），我们不能说“一个数学”
- 依照类，我们可以创建对象，这就是“实例化”
 - 现实世界中常称“对象”，程序世界中常称“实例”，
 - 二者并无太大区别，常常混用，初学者不必迷惑
- 使用new操作符创建类的实例
- 引用变量与实例的关系
 - 孩子与气球
 - 气球不一定有孩子牵着
 - 多个孩子可以使用各自的绳子牵着同一个气球，也可以都通过一根绳子牵着气球

```
using System;
namespace C_test
{
    class Test //类的申明
    {
    }
}
internal class Program
{
    static void Main(string[] args)
    {
        Test test = new Test(); //申明一个Test类的对象test
    }
}
```

三、类三大成员（属性、方法、事件）

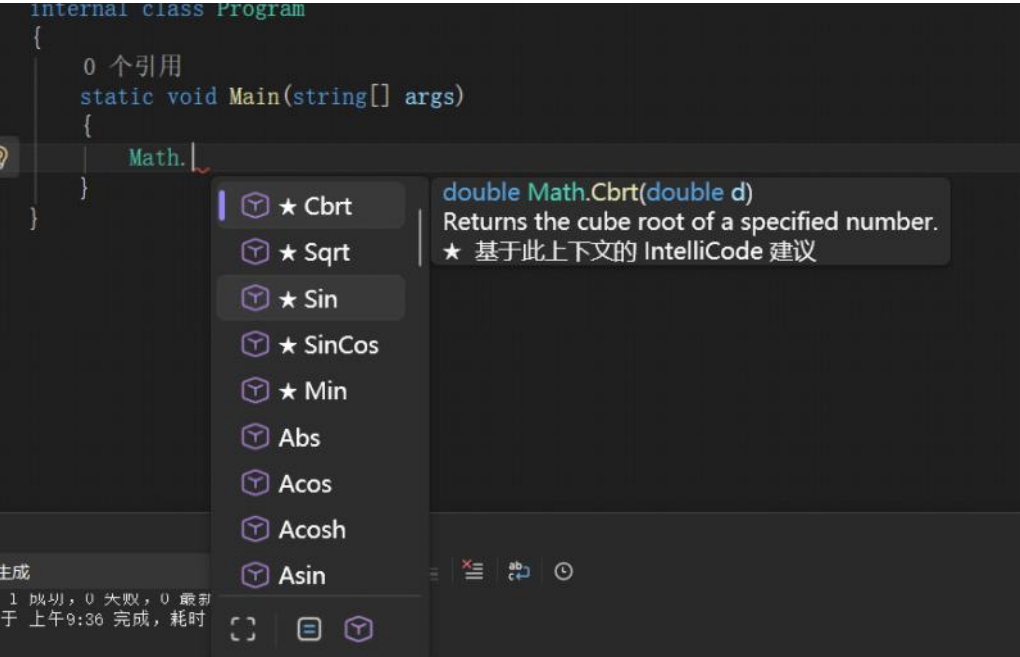
类的三大成员

- 属性（ Property ）
 - 存储数据，组合起来表示类或对象当前的状态
- 方法（ Method ）
 - 由C语言中的函数（ function ）进化而来，表示类或对象“能做什么”
 - 工作中90%的时间是在与方法打交道，因为它是“真正做事”、“构成逻辑”的成员
- 事件（ Event ）
 - 类或对象通知其它类或对象的机制，为C#所特有（ Java通过其它办法实现这个机制 ）
 - 善用事件机制非常重要
- 使用MSDN文档
- 某些特殊类或对象在成员方面侧重点不同
 - 模型类或对象重在属性，如Entity Framework
 - 工具类或对象重在方法，如Math， Console
 - 通知类或对象重在事件，如各种Timer

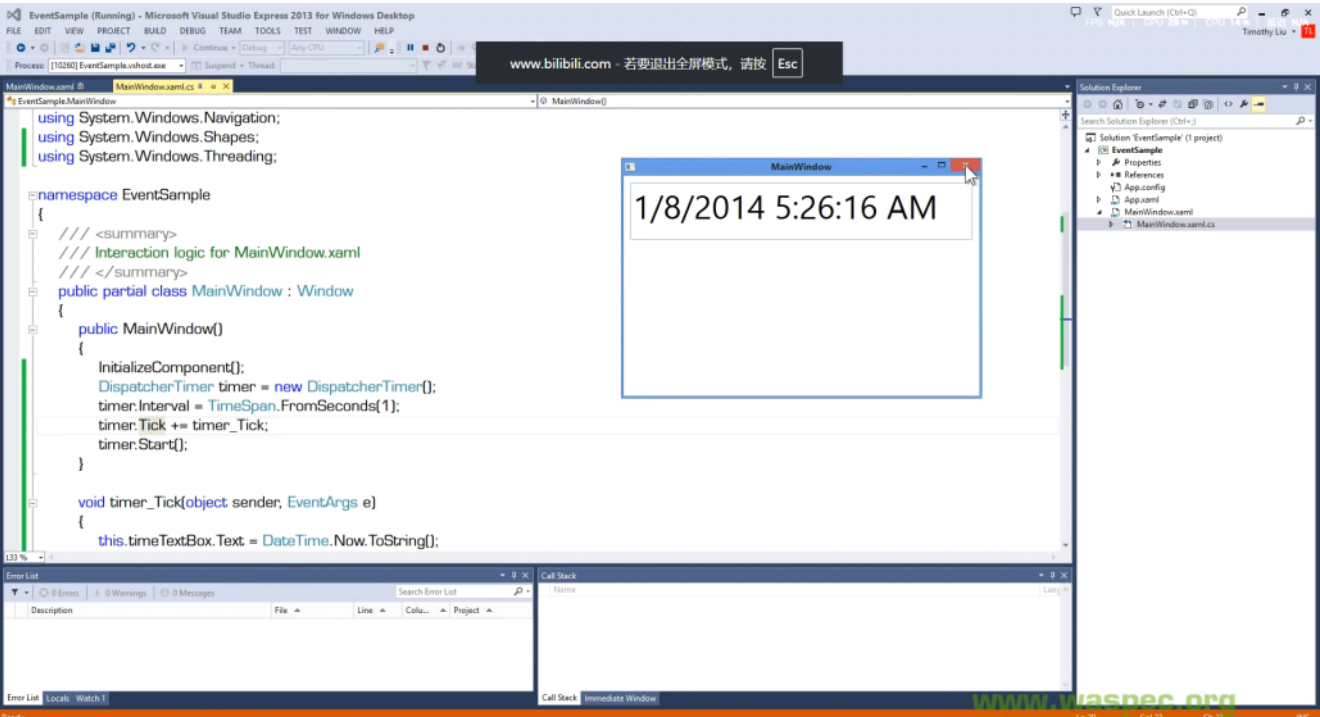
MSDN文档也就是Visual studio的官方文档，现在可以直接访问Vs的官网 <https://learn.microsoft.com/zh-cn/visualstudio/ide/?view=vs-2022>。

也可以在VS里面选中对应的类或者方法然后按F1就可以跳转到对应的文档。当然现在也可以用AI。

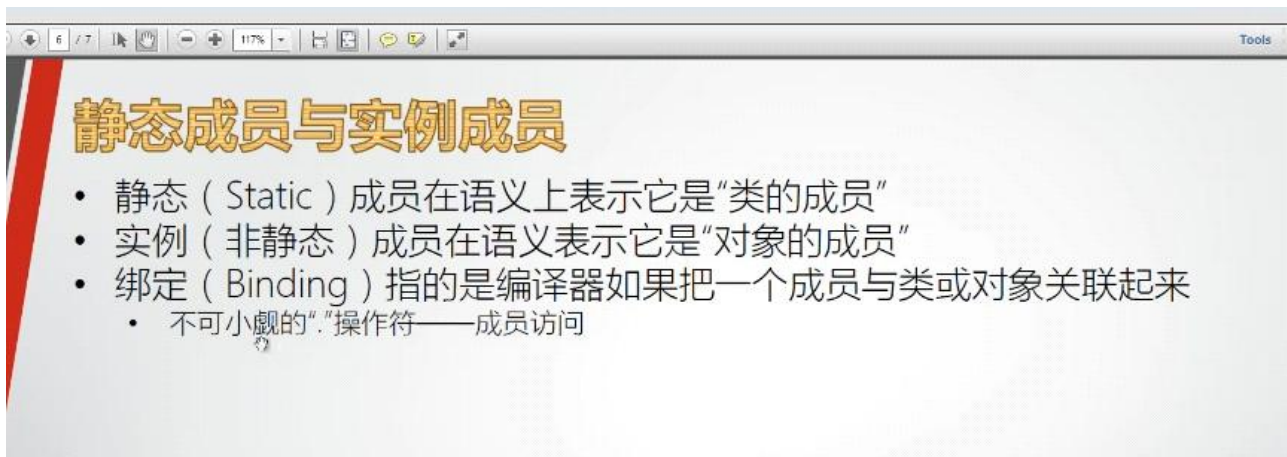
有些特殊的类和对象也有侧重点，比如Entity Framework类重属性，一个固定的Button按键这种就是几乎只有属性。也有重方法的，例如Math里面都是一些数学计算的方法。



也有重事件的，比如WPF里面的Timer。



四、静态成员和实例成员



可以换一个角度看，静态Static成员表示一种普遍现象，可以看成“类的成员”，例如人类这个类的最大年龄是150。

实例在语义上表示它是“对象的成员”，比如小明的年龄为18岁。

```
0 个引用
class Person { //声明一个类表示人类（抽象概念不表示具体的人）

    //public表示公共权限，类外的方法可以通过打点直接访问到
    public string name; //人类的姓名
    public int age;      //人类的年龄
    //private表示私有权限，类外只能通过本类的成员方法来访问到
    private int id; //身份证号
    //protected表示保护权限，和类的继承有关
    protected bool sex; //性别

    //类中加上static表示普遍现象，不代表个体，例如人类的年龄最大不超过150岁
    public static int MaxAge=150;

}
```

五、构成C#的基本元素

构成C#语言的基本元素

- 关键字 (Keyword)
 - 操作符 (Operator)
 - 标识符 (Identifier)
 - 什么是合法的标识符
 - 怎样阅读语言定义文档
 - 大小写规范
 - 命名规范
 - 标点符号
 - 文本 (字面值)
 - 整数
 - 多种后缀
 - 实数
 - 多种后缀
 - 字符
 - 字符串
 - 布尔
 - 空 (null)
 - 注释与空白
 - 单行
 - 多行 (块注释)
- 标记 (Token)

关键字是人类文字里挑出来能表达逻辑的那几个单词，每个单词都有自己独特的逻辑功能。

C# Keywords

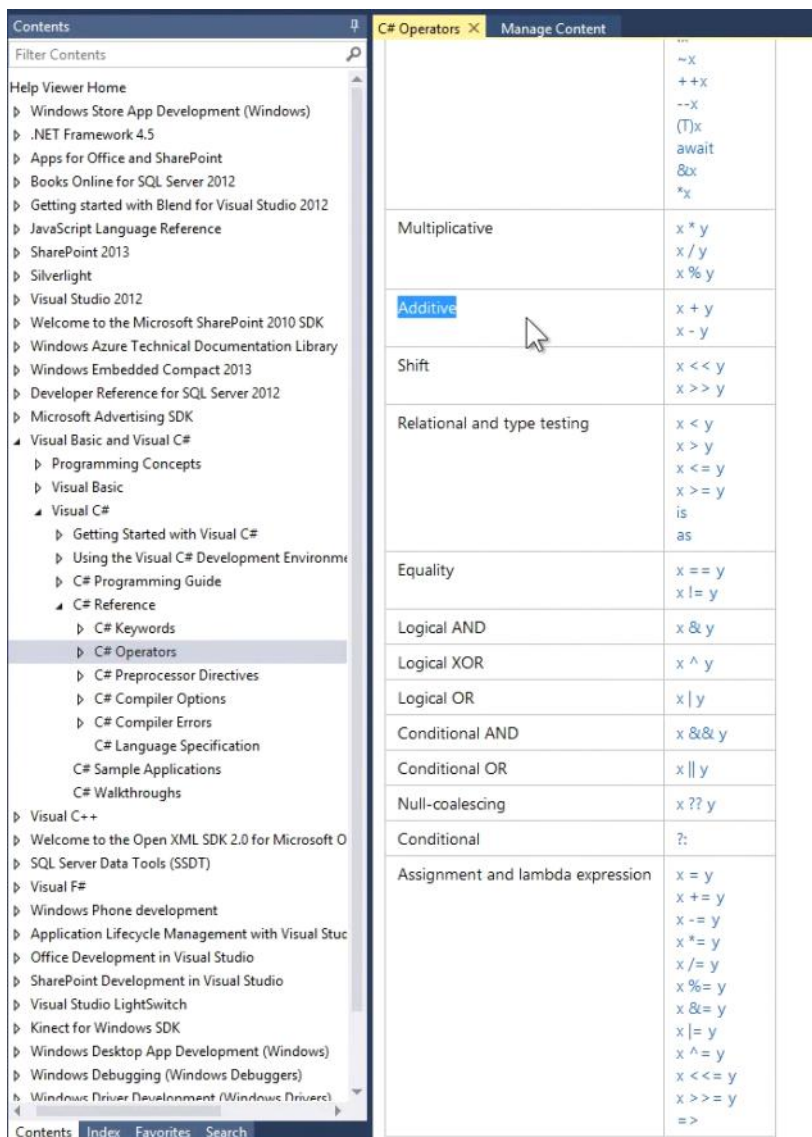
[Send Feedback](#) on this topic to Microsoft.

Keywords are predefined, reserved identifiers that have special meanings to the compiler. They cannot be used as identifiers in any part of a C# program.

The first table in this topic lists keywords that are reserved identifiers in any part of a C# program. The second table lists identifiers that are reserved for use as identifiers outside that context. Generally, as new keywords are added to the C# language, they are added to both tables.

abstract	as	base	bool
break	byte	case	catch
char	checked	class	const
continue	decimal	default	delegate
do	double	else	enum
event	explicit	extern	false
finally	fixed	float	for
foreach	goto	if	implicit
in	in (generic modifier)	int	interface
internal	is	lock	long
namespace	new	null	object
operator	out	out (generic modifier)	override
params	private	protected	public
readonly	ref	return	sbyte
sealed	short	sizeof	stackalloc
static	string	struct	switch
this	throw	true	try
typeof	uint	ulong	unchecked
unsafe	ushort	using	virtual
void	volatile	while	

操作符是变量之间用来运算的，分为算术运算符和逻辑运算符。



标识符是程序员自己取的名字，可以表示变量名、类名、对象名等，合法的标识符由字母数字下划线组成（数字不能开头），不能和关键字冲突。标识符取名时建议采用驼峰命名法。

```
int studentId;
int studentScore;
int student_id;
int student_score;
```

标点符号不参与运算，只代表一个符号

常用文本（变量值）：

```
static void Main(string[] args)
{
    int num1 = 20;
    long num2 = 1000000;
    short num3 = -1;           //有符号整数

    uint num4 = 121;
    ulong num5 = 12;
    ushort num6 = 12;         //无符号整数

    float f = 1.21F;           //单精度浮点
    double num4 = 21.1275212;  //双精度浮点

    decimal price = 19.99M;    // 金钱类的用这个
    bool isSunny = false;      //布尔类

    char c = 'a';              //字符
    string str = "217sjygdadkw"; //字符串
}
```

```
var i = 12;           //var表示自动类型变量，会根据赋值来给出对应的变量类型，不建议使用因为可读性不好
var f = 219.31F;
var d = 12.31232;
```

```
int a = 3;
int b = 4;
Console.WriteLine(a + b); //输出a变量+b变量
Console.WriteLine("a+b=result"); //只输出字符串
int result = a + b;
Console.WriteLine("{0}+{1}={2}", a, b, result); //类似于C语言的printf，0、1、2表示顺序，双引号后面的是对应的变量
Console.WriteLine($"{a}+{b}={result}"); //美元符号表示字符串插值模式，这样可以直接输出变量值
```

注释符号和C语言是一样的：//和/* */

初识类型、变量和方法

- 初识类型 (Type)
 - 亦称数据类型 (Data Type)
- 变量是存放数据的地方，简称“数据”
 - 变量的声明
 - 变量的使用
- 方法 (旧称函数) 是处理数据的逻辑，又称“算法”
 - 方法的声明
 - 方法的调用
- 程序 = 数据 + 算法
 - 有了变量和方法就可以写有意义的程序了

算法简介

- 循环初体验
- 递归初体验
- 计算1到100的和

```
using System;
namespace C_test
{
```



```

class Test //类的申明
{
    public void Fun(int x) //输入x，输出0到x的所有值
    {
        for (int i = 0; i < x; i++)
        {
            Console.WriteLine(i);
        }
    }
    public int Sum1(int x) //递归执行求和
    {
        if (x == 1) return 1;
        else
        {
            int result = x + Sum1(x-1);
            return result;
        }
    }
}

internal class Program
{
    static void Main(string[] args)
    {
        Test test = new Test();
        test.Fun(1000);
    }
}

```

六、类型、变量与对象

本节内容

- 什么是类型 (Type)
- 类型在C#语言中的作用
- C#语言的类型系统
- 变量、对象与内存

什么是类型 (Type)

- 又名数据类型 (Data Type)
 - A data type is a homogeneous collection of values, effectively presented, equipped with a set of operations which manipulate these values.
 - 是数据在内存中存储时的“型号”
 - 小内存容纳大尺寸数据会丢失精确度、发生错误
 - 大内存容纳小尺寸数据会导致浪费
 - 编程语言的数据类型与数学中的数据类型不完全相同
- 强类型语言与弱类型语言的比较
 - C语言示例：if条件
 - JavaScript示例：动态类型
 - C#语言对弱类型/动态类型的模仿

```

internal class Program
{
    static void Main(string[] args)
    {
        int x = 100;
    }
}

```

```

        long y = 100L;    //L结尾表示long类型
        x = y;    //报错，因为类型表示范围不一致
    }
}

```

- 一个C#类型中所包含的信息有：
 - 存储此类型变量所需的内存空间大小
 - 此类型的值可表示的最大、最小值范围
 - 此类型所包含的成员（如方法、属性、事件等）
 - 此类型由何基类派生而来
 - 程序运行的时候，此类型的变量在分配在内存的什么位置
 - Stack简介
 - Stack overflow
 - Heap简介
 - 使用Performance Monitor查看进程的堆内存使用量
 - 关于内存泄漏
 - 此类型所允许的操作（运算）

各类型的表示范围如下：

Type	Range	Size
sbyte	-128 to 127	Signed 8-bit integer
byte	0 to 255	Unsigned 8-bit integer
char	U+0000 to U+ffff	Unicode 16-bit character
short	-32,768 to 32,767	Signed 16-bit integer
ushort	0 to 65,535	Unsigned 16-bit integer
int	-2,147,483,648 to 2,147,483,647	Signed 32-bit integer
uint	0 to 4,294,967,295	Unsigned 32-bit integer
long	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Signed 64-bit integer
ulong	0 to 18,446,744,073,709,551,615	Unsigned 64-bit integer

If the value represented by an integer literal exceeds the range of **ulong**, a compilation error will occur.

The following table shows the precision and approximate ranges for the floating-point types.

Type	Approximate range	Precision
float	$\pm 1.5\text{e}-45$ to $\pm 3.4\text{e}38$	7 digits
double	$\pm 5.0\text{e}-324$ to $\pm 1.7\text{e}308$	15-16 digits

一个类型本质上可以用一个类来定义，所以包括方法、属性、事件等成员,也可以用上继承多态等。

```

internal class Program
{
    static void Main(string[] args)
    {
        Type thisType= typeof(int);    //Type表示类型，变量用来获取int的数据类型
        Console.WriteLine(thisType.Name);    //输出int型的类型名字
        Console.WriteLine(thisType.BaseType.FullName);    //输出int型的父类全名
        Console.WriteLine(thisType.BaseType.FullName.FullName);    //输出int型的父类的父类全名（报错
        //因为往上没有父类了）
    }
}

```

一个类型也具有允许该类的操作（运算），例如int类型支持单目自增++

静态的东西是装在硬盘上的，动态的东西是装在内存上的。内存里也包含堆和栈，栈主要是放调用的方法的，内存小速度快，堆主要发给，栈满了叫爆栈，堆满了叫内存泄露。

表格		
特性	栈 (Stack)	堆 (Heap)
内存分配	系统自动分配 / 释放，速度极快	手动 (C# 中由 GC) 分配 / 释放，速度慢
内存大小	小 (通常 MB 级别)，固定大小	大 (GB 级别)，动态扩展
存储内容	值类型、引用类型的引用 (指针)、方法调用栈帧	引用类型的实际数据、装箱后的值类型
访问方式	按值直接访问	按引用 (地址) 间接访问
内存碎片	无 (先进后出，连续分配)	有 (频繁分配 / 释放会产生碎片)
生命周期	随方法执行结束自动销毁	直到 GC 检测到无引用后回收
分配顺序	先进后出 (FILO)	无序分配

```
using System;

class Program
{
    // 自定义引用类型 (类)
    class Person // 引用类型，实际数据存在堆上
    {
        public int Age; // 值类型，作为引用类型的成员，也存在堆上
    }

    static void Main()
    {
        // 1. 值类型 (int) — 直接存在栈上
        int num = 10;

        // 2. 引用类型 (Person) — 分两部分：
        //    - 变量 person (引用/指针) 存在栈上
        //    - new Person() 的实际对象存在堆上
        Person person = new Person();
        person.Age = 20;

        // 3. 方法调用 — 方法的参数/局部变量入栈，方法结束后出栈
        ChangeValue(num);
        ChangePerson(person);

        Console.WriteLine("num = " + num); // 输出 10 (栈上值未被修改)
        Console.WriteLine("person.Age = " + person.Age); // 输出 30 (堆上数据被修改)
    }

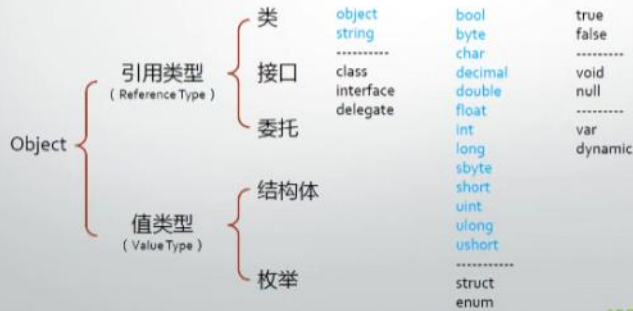
    static void ChangeValue(int value)
    {
        value = 100; // 修改的是栈上的副本，不影响原变量
    }

    static void ChangePerson(Person p)
    {
        p.Age = 30; // p 是栈上的引用，指向堆上的对象，修改的是堆上的实际数据
    }
}
```

面试题：C# 有哪些数据类型，回答的时候能答这五大类型就可以了

C#语言的类型系统

- C#的五大数据类型
 - 类 (Classes) : 如Window, Form, Console, String
 - 结构体 (Structures) : 如Int32, Int64, Single, Double
 - 枚举 (Enumerations) : 如HorizontalAlignment, Visibility
 - 接口 (Interfaces)
 - 委托 (Delegates)
- C#类型的派生谱系



C#的数据类型可以归为5类，最基础的就是类类型class、结构体类型struct、枚举类型enum

变量、对象与内存

- 什么是变量
 - 表面上来看（从C#代码的上下文行文上来看），变量的用途是存储数据
 - 实际上，变量表示了存储位置，并且每个变量都有一个类型，以决定什么样的值能够存入变量
 - 变量一共有7种
 - 静态变量，实例变量（成员变量，字段），数组元素，值参数，引用参数，输出形参，局部变量
 - 狭义的变量指局部变量，因为其它种类的变量都有自己的约定名称
 - 简单地讲，局部变量就是方法体（函数体）里声明的变量
 - 变量的声明
 - 有效的修饰符组合_{opt} 类型 变量名 初始化器_{opt}
- 值类型的变量
 - 以byte/sbyte/short/ushort为例
 - 值类型没有实例，所谓的“实例”与变量合而为一
- 引用类型的变量与实例
 - 引用类型变量与实例的关系：引用类型变量里存储的数据是对象的内存地址
- 局部变量是在stack上分配内存
- 变量的默认值
- 常量（值不可改变的变量）
- 装箱与拆箱（Boxing & Unboxing）

变量 = 以变量名所对应的内存地址为起点、以其数据类型所要求的存储空间为长度的一块内存区域

以下是值类型和引用类型的区别：

表格		
特性	值类型 (Value Type)	引用类型 (Reference Type)
内存存储位置	默认在栈上（除非作为引用类型成员存储堆上）	引用（指针）在栈上，实际数据在堆上
数据存储方式	直接存储实际值	存储堆上数据的地址（引用）
赋值 / 传参行为	复制整个值（值传递），新旧变量互不影响	复制引用地址（引用传递），新旧变量指向同一堆对象
默认值	有明确默认值（如 int=0、bool=false）	默认值为 null（无引用）
内存管理	栈自动释放，无需 GC	堆上数据由 GC 回收
类型举例	int、float、bool、struct、enum、char	class、interface、delegate、string、数组
是否可为 null	非可空值类型不可为 null（C#8 + 可空值类型 int? 除外）	天然可 null（除非加 nonnull 约束）

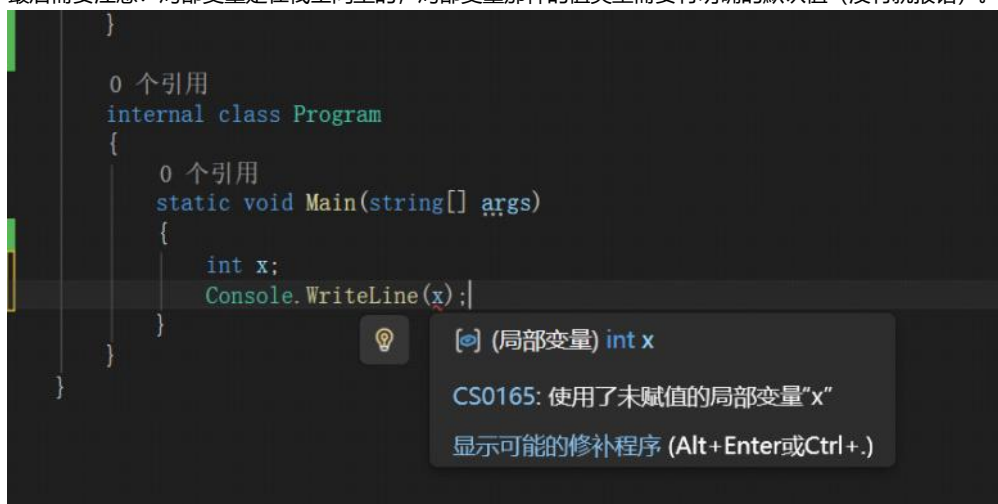

```

namespace TypeInCSharp
{
    class Program
    {
        static void Main(string[] args)
        {
            Student stu; 引用类型
            stu = new Student();
            Student stu2; I
            stu2 = stu; stu将地址复制给stu2, 因此两个变量指向相同的内存空间
        }
    }

    class Student
    {
        uint ID;
        ushort Score;
    }
}

```

最后需要注意：局部变量是在栈空间里的，局部变量那种的值类型需要有明确的默认值（没有就报错）。引用的默认值是0



```

namespace TypeInCSharp
{
    class Program
    {
        static void Main(string[] args)
        {
            Student stu = new Student(); 分配好后没赋值
            Console.WriteLine(stu.ID);
            Console.WriteLine(stu.Score); 默认值是0
        }
    }

    class Student
    {
        public uint ID;
        public ushort Score; I
    }
}

```

常量就是定义的时候必须赋值，然后赋值后不能修改。

装箱和拆箱说白了可以总结成一句话：拆箱：堆到栈 装箱：栈到堆，尽量避免装箱来提高性能

```
namespace TypeInCSharp
{
    class Program
    {
        static void Main(string[] args)
        {
            int x = 100; 装箱
            object obj;
            obj = x;
            int y = (int)obj; 拆箱
            Console.WriteLine(y);
        }
    }

    class Student
    {
        public uint ID;
        public ushort Score;
    }
}
```

七、方法

1、方法的由来

方法的由来

- 方法（method）的前身是C/C++语言的函数（function）
 - 方法是面向对象范畴的概念，在非面向对象语言中仍然称为函数
 - 使用C/C++语言做对比
- 永远都是类（或结构体）的成员
 - C#语言中函数不可能独立于类（或结构体）之外
 - 只有作为类（结构体）的成员时才被称为方法
 - C++中是可以的，称为“全局函数”
- 是类（或结构体）最基本的成员之一
 - 最基本的成员只有两个——字段与方法（成员变量与成员方法），本质还是数据+算法
 - 方法表示类（或结构体）“能做什么事情”
- 为什么需要方法和函数
 - 目的1：隐藏复杂的逻辑
 - 目的2：复用（reuse，重用）
 - 示例：计算圆面积、圆柱体积、圆锥体积

最早的C语言是面向过程的语言，C++是面向对象+面向过程的语言，C#是纯面向对象的语言。

注意C#中的方法必须定义在类里面，C/C++里面的全局函数是不行的。

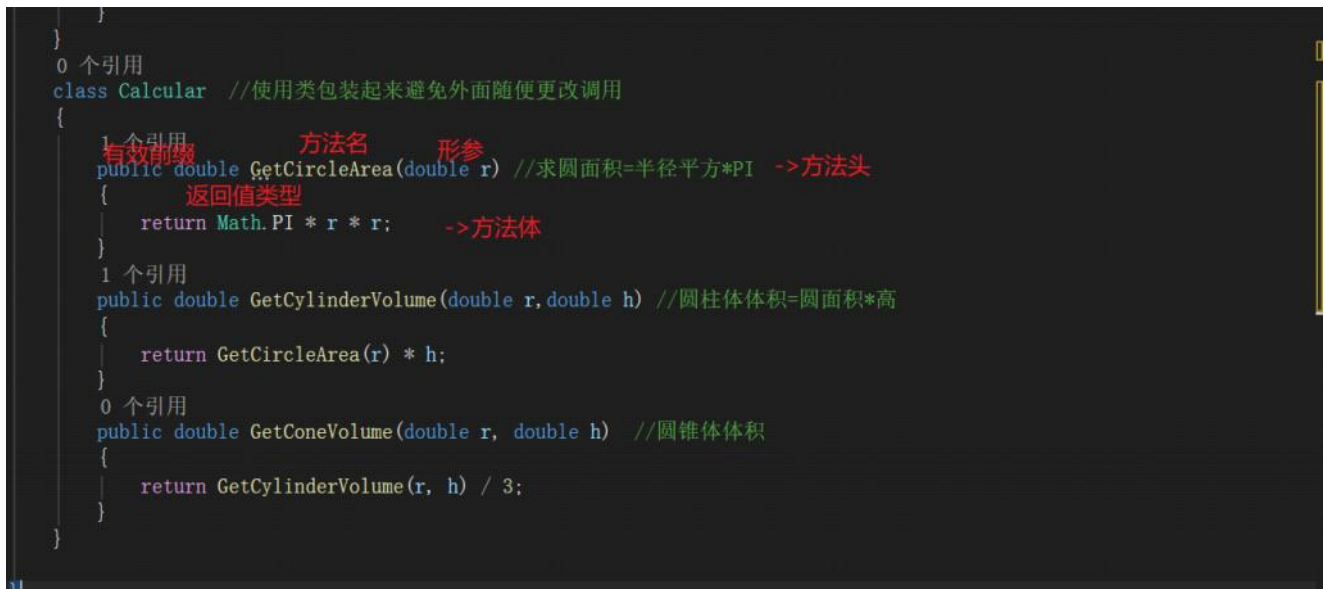
类的最基本成员就是属性和方法。

```
class Calcular //使用类包装起来避免外面随便更改调用
{
    public double GetCircleArea(double r) //求圆面积=半径平方*PI
```

```

    {
        return Math.PI * r * r;
    }
    public double GetCylinderVolume(double r, double h) //圆柱体体积=圆面积*高
    {
        return GetCircleArea(r) * h;
    }
    public double GetConeVolume(double r, double h) //圆锥体体积
    {
        return GetCylinderVolume(r, h) / 3;
    }
}

```



2、方法的声明和调用

方法的声明与调用

- 声明方法的语法详解
 - 参见C#语言文档（声明/定义不分家）
 - Parameter全称为“formal parameter”形式上的参数，简称“形参”
 - Parameter是一种变量
- 为方的命名规范
 - 大小写规范
 - 需要以动词或者动词短语作为名字
- 重温静态（static）方法和实例方法
- 调用方法
 - Argument中文C#文档的官方译法为“实际参数”，简称“实参”，可理解为调用方法时的真实条件
 - 调用方法时的argument列表要与定义方法时的parameter列表相匹配
 - C#是强类型语言，argument是值、parameter是变量，值与变量一定要匹配，不然编译器会报错

parameter 📄

美: [pəˈræmɪtər] 🗣 英: [pəˈræmɪtə(r)] 🗣

n. 范围；决定因素；规范

网络 参数；参量；指标参数

argument 📄

美: [ˈɑːrɡjʊmənt] 🗣 英: [ˈɑː(r)ɡjʊmənt] 🗣

n. 争论；论点；辩论；论据

网络 参数；论证；引数

注意区分方法的声明和调用，声明的是形参，调用的是实参。

```

using System;
namespace C_test
{

```

```

internal class Program
{
    static void Main(string[] args)
    {
        Calculat.GetCircleArea(10); //静态方法和类绑定
        Calculat calculat = new Calculat();
        calculat.GerCircleArea(10); //静态方法不和对象绑定，会报错
    }
}

class Calculat
{
    public static double GetCircleArea(double r) //静态方法和类绑定
    {
        return Math.PI * r * r;
    }
}

```

3、构造器

构造器

- 构造器 (constructor) 是类型的成员之一
- 狭义的构造器指的是“实例构造器” (instance constructor)
- 如何调用构造器
- 声明构造器
- 构造器的内存原理

默认构造器举例：

```

using System;
namespace C_test
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Student student = new Student(); //默认构造器在new一个对象的时候起作用，给对象的数据成员赋
            初始值
        }
    }

    class Student //该类里面没有构造器，将使用自带的默认构造器
    {
        public int Id;
        public string Name;
    }
}

```

带参数构造器和不带参数的构造器举例：

```

using System;
using System.Security.Cryptography.X509Certificates;
namespace C_test
{
    internal class Program
    {

```



```

static void Main(string[] args)
{
    Student student1 = new Student(); //将用不带参数的构造器
    Student student2 = new Student(121, "小明"); //将用待参数的构造器
    Console.WriteLine(student1.Id);
    Console.WriteLine(student1.Name);
    Console.WriteLine(student2.Id);
    Console.WriteLine(student2.Name);
}
}
class Student
{
    public Student() //创建一个自定义不带参数的构造器，当new一个对象的时候没填信息，将会使用这个默认信息
    {
        this.Id = 1; //this 就是表示这个函数中所指的变量
        this.Name="Default";
    }
    public Student(int id, string name) //创建一个自定义带参数的构造器，new一个对象的时候填信息就会用这个
    {
        Id = id;
        Name = name;
    }
    public int Id;
    public string Name;
}
}

```

4、方法的重载

方法的重载 (Overload)

- 调用重载方法的示例
- 声明带有重载的方法
 - 方法签名 (method signature) 由方法的名称、类型形参的个数和它的每一个形参 (按从左到右的顺序) 的类型和种类 (值、引用或输出) 组成。**方法签名不包含返回类型。**
 - 实例构造函数签名由它的每一个形参 (按从左到右的顺序) 的类型和种类 (值、引用或输出) 组成。
 - 重载决策 (到底调用哪一个重载) : 用于在给定了参数列表和一组候选函数成员的情况下，选择一个最佳函数成员来实施调用。

方法重载说白了就是：方法名称相同，返回值不同，形参是不同的，在调用方法的时候会在这些同名方法中一个一个的比较数据类型和形参数量。

```

using System;
using System.Security.Cryptography.X509Certificates;
namespace C_test
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Calculator calculator = new Calculator();
            Console.WriteLine(calculator.Add(1, 2)); //调用第一个
        }
    }
}

```

```

        Console.WriteLine(calculator.Add(1D, 2D)); //调用第二个
        Console.WriteLine(calculator.Add(1, 2, 3)); //调用第三个
    }
}
class Calculator
{
    //在调用Add方法的时候会在这些同名方法中一个一个的比较数据类型和形参数量
    public int Add(int a, int b)
    {
        return a + b;
    }
    public double Add(double a, double b)
    {
        return a + b;
    }
    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}

```

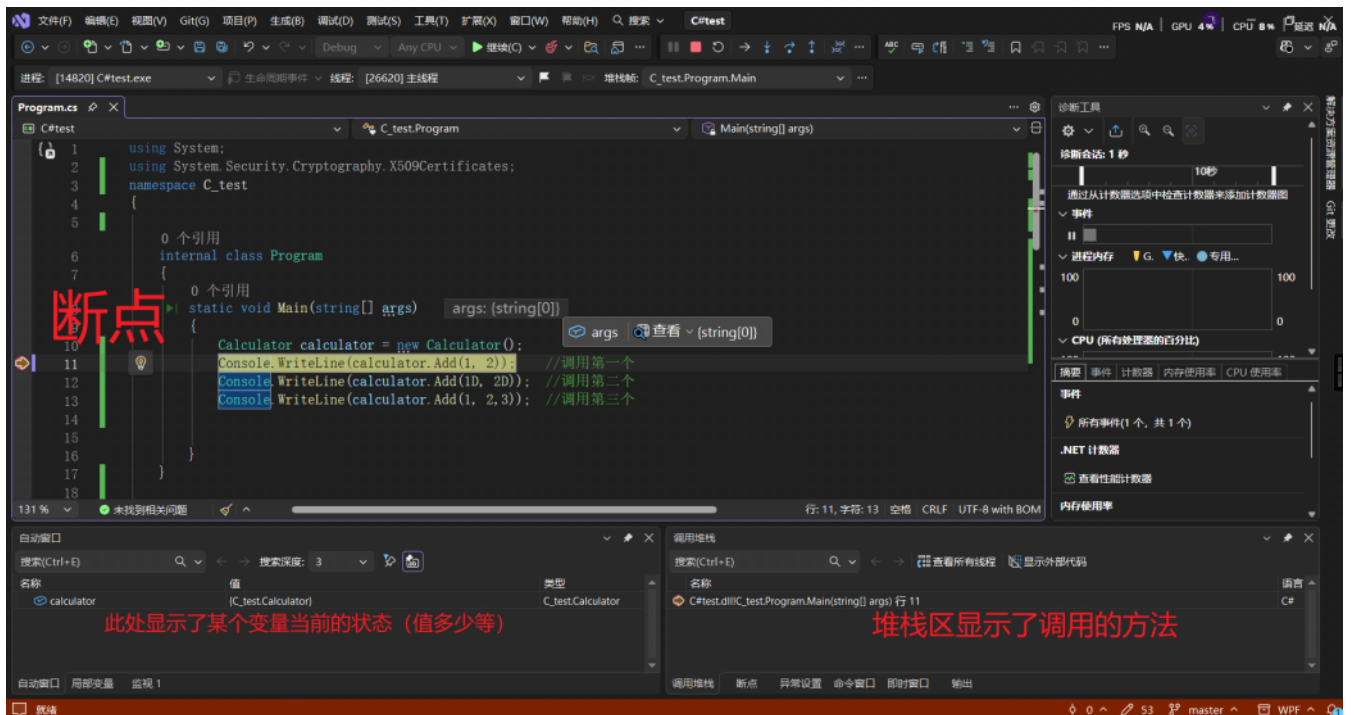
5、VS里Debug的教程

如何对方法进行debug

- 设置断点 (breakpoint)
- 观察方法调用时的call stack
- Step-in, Step-over, Step-out
- 观察局部变量的值与变化

断点就是程序运行到这里的时候会停下来的点，方便程序员观察。

在VS中的代码左侧按F9就可以设置一个断点。



Step-in方式的debug就是一行一行的执行，按F11可以实现
Step-over方式的debug就是当前的方法直接跳过，一段一段的执行，这个可以按F10
Step-out立即执行完当前所在函数的剩余代码，直接跳出到调用这个函数的上一级代码行并暂停。

表格			
操作	快捷键	核心行为	适用场景
Step-in（步入）	F11	逐行执行，遇到函数会进入内部	想查看函数内部具体逻辑
Step-over（步过）	F10	逐行执行，遇到函数直接执行完（不进入）	只关注当前层级代码，跳过函数
Step-out（步出）	Shift+F11	执行完当前函数剩余代码，跳回调用处	进入函数后想快速跳出

八、操作符

语句=操作符+表达式

1、操作符概览

类别	运算符
基本	x.y f(x) a[x] x++ x-- new typeof default checked unchecked delegate sizeof ->
一元	+ - ! ~ ++x --x (T)x await &x *x
乘法	* / %
加减	+ -
移位	<< >>
关系和类型检测	< > <= >= is as
相等	== !=
逻辑“与”	&
逻辑 XOR	^
逻辑 OR	
条件 AND	&&
条件 OR	
null 合并	??
条件	?:
赋值和 lambda 表达式	= *= /= %= += -= <<= >>= &= ^= = ==>

- 操作符 (Operator) 也译为“运算符”
- 操作符是用来操作数据的，被操作符操作的数据称为操作数 (Operand)

以上的运算符应该都很熟悉了。

2、操作符的本质

操作符的本质

- 操作符的本质是函数 (即算法) 的“简记法”
 - 假如没有发明“+”、只有Add函数，算式 $3+4+5$ 将可以写成`Add(Add(3,4),5)`
 - 假如没有发明“ $\times$ ”、只有Mul函数，那么算式 $3+4\times 5$ 将只能写成`Add(3,Mul(4,5))`，注意优先级
- 操作符不能脱离与它关联的数据类型
 - 可以说操作符就是与固定数据类型相关联的一套基本算法的简记法
 - 示例：为自定义数据类型创建操作符

www.waspec.org

操作符不能脱离与它关联的数据类型。

```
internal class Program
{
    static void Main(string[] args)
    {
        int x1 = 5;
        int y1 = 2;
        double z1 = x1 / y1;
        Console.WriteLine(z1); //输出2
    }
}

double x2 = 5;
double y2 = 2;
```



```

        double x3 = x2 / y2;
        Console.WriteLine(x3); //输出2.5
    }
}

```

为自定义类型创造操作符:

```

using System;
using System.Security.Cryptography.X509Certificates;
namespace C_test
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Person person1 = new Person();
            Person person2 = new Person();
            person1.Name = "jack";
            person2.Name = "cherry";
            List<Person> nation = Person.Getmary(person1, person2); //建立国家链表的时候jack和cherry进入链表中并生了11个孩子
            foreach(var p in nation) //遍历链表
            {
                Console.WriteLine(p.Name);
            }
        }
    }
}

class Person
{
    public string Name;
    public static List<Person>Getmary(Person p1, Person p2)
    {
        List<Person> people = new List<Person>(); //建立链表
        people.Add(p1); //将夫妻两人放入链表中
        people.Add(p2);
        for(int i = 0; i < 11; i++)
        {
            Person child = new Person(); //夫妻生11个孩子
            child.Name = p1.Name + "&" + p2.Name + "s child";
            people.Add(child);
        }
        return people;
    }
}

```

3、优先级与运算顺序

优先级与运算顺序

- 操作符的优先级
 - 可以使用圆括号提高被括起来表达式的优先级
 - 圆括号可以嵌套
 - 不像数学里有方括号和花括号，在C#语言里“[]”与“{}”有专门的用途
- 同优先级操作符的运算顺序
 - 除了带有赋值功能的操作符，同优先级操作符都是由左向右进行运算
 - 带有赋值功能的操作符的运算顺序是由右向左
 - 与数学运算不同，计算机语言的同优先级运算没有“结合率”
 - 3+4+5只能理解为Add(Add(3,4),5)不能理解为Add(3,Add(4,5))

```
int x=100;
x = 3 + 6 + 4; //从左向右算
int y = 100;
x += y;        //带有赋值功能
int z = 100;
x += y += z;   //从右往左，先执行y+z然后+x
```

4、各类操作符的示例

类别	运算符
基本	x.y f(x) a[x] x++ x-- new typeof default checked unchecked delegate sizeof ->
一元	+ - ! ~ ++x --x (T)x await &x *x
乘法	* / %
加法	+ - ?
移位	<< >>
关系和类型检测	< > <= >= is as
相等	== !=
逻辑“与”	&
逻辑 XOR	^
逻辑 OR	
条件 AND	&&
条件 OR	
null 合并	??
条件	?:
赋值和 lambda 表达式	= *= /= %= += -= <<= >>= &= ^= = =>

上图的操作符优先级从上往下依次下降。

基本操作符：x.y表示成员访问符、f(x)表示方法调用符、a[x]表示访问数组的元素、x++和x--表示后置的自增和自减、new表示声明对象、typeof帮助查看数据类型的基本结构（例如Type t = typeof(int)是获取int类型的各种信息）、default帮助我们获得默认值（例如int x = default(int)，对应内存区域默认刷成0）、checked和unchecked表示该数据类型有没有溢出、delegate表示委托（目前过时了，被lamuba语法取代）、sizeof返回类型所占字节大小、箭头->表示访问结构体内指针指向的值

```

1 using System;
2 using System.Windows.Forms;
3
4 namespace OperatorsExample
5 {
6     class Program
7     {
8         static void Main(string[] args)
9         {
10             var myForm = new Form() { Text = "Hello" };
11             var person = new { Name = "Mr.Okay", Age = 34 };
12             Console.WriteLine(person.Name);
13             Console.WriteLine(person.Age);
14             Console.WriteLine(person.GetType().Name);
15         }
16     }
17 }

```

```

7 static void Main(string[] args)
8 {
9     uint x = uint.MaxValue;
10    Console.WriteLine(x);
11    string binStr = Convert.ToString(x, 2);
12    Console.WriteLine(binStr);
13    checked
14    {
15        try
16        {
17            uint y = x + 1;
18            Console.WriteLine(y);
19        }
20        catch (OverflowException ex)
21        {
22            Console.WriteLine("There's overflow!");
23        }
24    }
25 }
26 }
27 }

```

一元操作符：首先&是取地址操作符、*表示取值操作符，需要在unsafe{}代码块里面。+-单目就是单纯的给数字加正负号、!表示逻辑非、~表示异或，就是将数字表示的二进制值都按位取反、前++--表示先自增自建后执行语句、(T)表示强制类型转换操作符、

```

int x = -100; //负的
Console.WriteLine(-x); //负负得正

```

乘法、加法、移位操作符：无需多言，稍微难点的移位符就是将二进制数左移或者右移一位，左移相当于*2，右移相当于/2。

关系操作符：所有的关系符的运算结果都是bool类型的，数字之间是比較大小，字符之间是比較ASCII码的大小。

逻辑操作符：与或非无需多言，注意短路效应，例如x>y && a++>3的时候，如果x>y不成立的话，后面的就不会再看了，a++不会执行。

二进制操作符：按位与、按位或、按位异或这种都是操作二进制数的，例如x&y是两者二进制数相与。

条件表达式：这个是唯一一个三位运算符，例如x>y ? A : B，这个相当于简化版的if else分支，如果x>y则执行A，如果x<=y则执行B

赋值和lambda表达式：注意带赋值符号则从右往左，lambda表达式后面再说。

九、类型转换

类型转换

- 隐式(implicit)类型转换
 - 不丢失精度的转换
 - 子类向父类的转换
 - 装箱
- 显式(explicit)类型转换
 - 有可能丢失精度（甚至发生错误）的转换，即cast
 - 拆箱
 - 使用Convert类
 - ToString方法与各数据类型的Parse/TryParse方法
- 自定义类型转换操作符
 - 示例

```
string str1 = Console.ReadLine();
string str2 = Console.ReadLine();
int x = Convert.ToInt32(str1);    //强制类型转换将输入的string转换成int
int y = Convert.ToInt32(str2);
Console.WriteLine(x+y);
```

1、隐式类型转换（不丢失精度）

隐式类型转换是编译器自动转换，不损失精度，在不同类型的值计算的时候会进行。

6.1.2 隐式数值转换

隐式数值转换为：

- 从 sbyte 到 short、int、long、float、double 或 decimal。
- 从 byte 到 short、ushort、int、uint、long、ulong、float、double 或 decimal。
- 从 short 到 int、long、float、double 或 decimal。
- 从 ushort 到 int、uint、long、ulong、float、double 或 decimal。
- 从 int 到 long、float、double 或 decimal。
- 从 uint 到 long、ulong、float、double 或 decimal。
- 从 long 到 float、double 或 decimal。
- 从 ulong 到 float、double 或 decimal。
- 从 char 到 ushort、int、uint、long、ulong、float、double 或 decimal。
- 从 float 到 double。

从 int、uint、long 或 ulong 到 float 的转换以及从 long 或 ulong 到 double 的转换可能导致精度损失，但决不会影响数值大小。其他的隐式数值转换决不会丢失任何信息。

不存在向 char 类型的隐式转换，因此其他整型的值不会自动转换为 char 类型。

隐式类型转换是通过多态来实现的，多态可以理解为子类向父类的隐式类型转换。

```
using System;
using System.Runtime.InteropServices.Marshalling;
using System.Security.Cryptography.X509Certificates;
namespace C_test
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Teacher teacher = new Teacher();
            Human human = teacher; //将teacher所存储的地址交给human（子类到父类的隐式转换）
            human.Eat(); //子类可以访问父类的方法
            human.Think(); //子类可以访问自己的方法
            human.teach(); //报错，子类不能访问自己子类的方法
        }
    }
}

class Animal //动物类
{
    public void Eat()
    {
        Console.WriteLine("Eating...");
    }
}
```

```

    }

}
class Human : Animal //人类
{
    public void Think()
    {
        Console.WriteLine("思考....");
    }
}
class Teacher : Human //老师类
{
    public void Teach()
    {
        Console.WriteLine("教学.....");
    }
}
}

```

2、显式类型转换（可能导致精度丢失）

在对象或者变量前面加上括号就行了，使用显式类型转换代表程序员自愿承担精度丢失的后果。

6.2.1 显式数值转换

显式数值转换是指从一个 *numeric-type* 到另一个 *numeric-type* 的转换，此转换不能用已知的隐式数值转换（第 6.1.2 节）实现，它包括：

- 从 `sbyte` 到 `byte`、`ushort`、`uint`、`ulong` 或 `char`。
- 从 `byte` 到 `sbyte` 和 `char`。
- 从 `short` 到 `sbyte`、`byte`、`ushort`、`uint`、`ulong` 或 `char`。
- 从 `ushort` 到 `sbyte`、`byte`、`short` 或 `char`。
- 从 `int` 到 `sbyte`、`byte`、`short`、`ushort`、`uint`、`ulong` 或 `char`。
- 从 `uint` 到 `sbyte`、`byte`、`short`、`ushort`、`int` 或 `char`。
- 从 `long` 到 `sbyte`、`byte`、`short`、`ushort`、`int`、`uint`、`ulong` 或 `char`。
- 从 `ulong` 到 `sbyte`、`byte`、`short`、`ushort`、`int`、`uint`、`long` 或 `char`。
- 从 `char` 到 `sbyte`、`byte` 或 `short`。
- 从 `float` 到 `sbyte`、`byte`、`short`、`ushort`、`int`、`uint`、`long`、`ulong`、`char` 或 `decimal`。
- 从 `double` 到 `sbyte`、`byte`、`short`、`ushort`、`int`、`uint`、`long`、`ulong`、`char`、`float` 或 `decimal`。
- 从 `decimal` 到 `sbyte`、`byte`、`short`、`ushort`、`int`、`uint`、`long`、`ulong`、`char`、`float` 或 `double`。

由于显式转换包括所有隐式和显式数值转换，因此总是可以使用强制转换表达式（第 7.7.6 节）从任何 *numeric-type* 转换为任何其他的 *numeric-type*。

除了加括号，还有 `convert` 和 `toString` 和 `Parse` 这两种显式转换的方法。


```
MainWindow.xaml
MainWindow.xaml.cs
- btn_Click(object sender, RoutedEventArgs e)

13 using System.Windows.Navigation;
14 using System.Windows.Shapes;
15
16 namespace Convert
17 {
18     /// <summary>
19     /// Interaction logic for MainWindow.xaml
20     /// </summary>
21     public partial class MainWindow : Window
22     {
23         public MainWindow()
24         {
25             InitializeComponent();
26         }
27
28         private void btn_Click(object sender, RoutedEventArgs e)
29         {
30             double x = double.Parse(this.tb1.Text); 显式类型转换
31             double y = double.Parse(this.tb2.Text);
32             double result = x + y;
33             this.tb3.Text = result.ToString();
34         }
35     }
36 }
37
```

十、表达式

各类表达式概览

- C#语言中表达式的分类
 - A value. Every value has an associated type. 任何能得到值的运算（回顾操作符和结果类型）
 - A variable. Every variable has an associated type.
 - A namespace.
 - A type.
 - A method group. 例如：Console.WriteLine，这是一组方法，重载决策决定具体调用哪一个
 - A null literal.
 - An anonymous function.
 - A property access.
 - An event access.
 - An indexer access.
 - Nothing. 对返回值为void的方法的调用
- 复合表达式的求值
 - 注意操作符的优先级和同优先级操作符的运算方向
- 参考C#语言定义文档
 - 仅作参考，不必深究——毕竟我们是在学习语言、不是去实现这门语言

表达式语句如下：

语句详解

statement:
labeled-statement
declaration-statement
embedded-statement
embedded-statement:
block
empty-statement
expression-statement
selection-statement
iteration-statement
jump-statement
try-statement
checked-statement
unchecked-statement
lock-statement
using-statement
yield-statement

- 声明语句
- 表达式语句
- 块语句（简称“块”）
- 选择（判断、分支）语句
- 迭代（循环）语句
- 跳转语句
- try...catch...finally语句
- using语句
- yield语句
- checked/unchecked语句
- lock语句（用于多线程）
- 标签语句
- 空语句

练习使用code snippet

其中最重要的就是分支循环跳转异常语句，详细的如下：

If-else分支语句：

```
// 基本形式
int score = 85;
if (score >= 60)
{
    Console.WriteLine("及格"); // 条件为true时执行
}
// if-else 形式（二选一）
if (score >= 90)
{
    Console.WriteLine("优秀");
}
else
{
    Console.WriteLine("非优秀"); // 条件为false时执行
}
// if-else if-else 形式（多选一）
if (score >= 90)
{
    Console.WriteLine("优秀");
}
else if (score >= 80)
{
    Console.WriteLine("良好");
}
else if (score >= 60)
{
    Console.WriteLine("及格");
}
else
{
    Console.WriteLine("不及格"); // 所有条件都不满足时执行
}
```

switch语句：

```
int day = 3;
switch (day)
{
```

```

case 1:
    Console.WriteLine("周一");
    break; // 跳出switch, 避免执行后续case
case 2:
    Console.WriteLine("周二");
    break;
case 3:
    Console.WriteLine("周三");
    break;
default: // 所有case都不匹配时执行
    Console.WriteLine("无效日期");
    break;
}

```

for循环:

```

// 输出1-5
for (int i = 1; i <= 5; i++) // 初始化i=1 → 条件i<=5 → 执行循环体 → i++
{
    Console.WriteLine(i);
}

```

foreach迭代器:

```

string[] fruits = { "苹果", "香蕉", "橙子" };
// 遍历每一个元素
foreach (string fruit in fruits)
{
    Console.WriteLine(fruit);
}

```

while循环:

```

int count = 0;
// 条件为true时循环
while (count < 3)
{
    Console.WriteLine("循环中: " + count);
    count++; // 必须手动修改条件变量, 否则死循环
}

```

Do-while循环

```

int num = 5;
do
{
    Console.WriteLine("执行一次: " + num);
    num--;
} while (num > 10); // 条件为false, 但循环体已执行1次

```

break和continue语句:

```

// break示例: 输出1-3后终止
for (int i = 1; i <= 5; i++)
{
    if (i == 4) break;
    Console.WriteLine(i);
}
// continue示例: 跳过偶数, 只输出奇数
for (int i = 1; i <= 5; i++)
{
    if (i % 2 == 0) continue;
    Console.WriteLine(i);
}

```

异常处理语句try-catch-finally:

```

try
{

```

```

// 可能抛出异常的代码（受保护的代码）
int a = 10;
int b = 0;
int result = a / b; // 此处会抛出DivideByZeroException
Console.WriteLine(result);
}
catch (DivideByZeroException ex) // 捕获指定类型的异常
{
    // 异常处理逻辑（如提示用户、记录日志）
    Console.WriteLine("错误：除数不能为0 → " + ex.Message);
}
catch (Exception ex) // 捕获所有其他异常（父类异常，放最后）
{
    Console.WriteLine("未知错误 → " + ex.Message);
}
finally
{
    // 无论是否发生异常，都会执行的代码（如释放资源：关闭文件、数据库连接）
    Console.WriteLine("finally块执行（资源清理）");
}

```

抛出异常语句throw

```

void CheckAge(int age)
{
    if (age < 0)
    {
        // 主动抛出异常
        throw new ArgumentException("年龄不能为负数");
    }
    Console.WriteLine("年龄合法： " + age);
}
// 调用方法（需用try-catch捕获）
try
{
    CheckAge(-5);
}
catch (ArgumentException ex)
{
    Console.WriteLine(ex.Message);
}

```

十一、字段和属性

字段

- 什么是字段
 - 字段（field）是一种表示与对象或类型（类与结构体）关联的变量
 - 字段是类型的成员，旧称“成员变量”
 - 与对象关联的字段亦称“实例字段”
 - 与类型关联的字段称为“静态字段”，由static修饰
- 字段的声明
 - 参见C#语言定义文档
 - 尽管字段声明带有分号，但它不是语句
 - 字段的名字一定是名词
- 字段的初始值
 - 无显式初始化时，字段获得其类型的默认值，所以字段“永远都不会未被初始化”
 - 实例字段初始化的时机——对象创建时
 - 静态字段初始化的时机——类型被加载（load）时
- 只读字段
 - 实例只读字段
 - 静态只读字段

字段还有个名字就叫成员变量，是类的直接数据成员，是变量的一种，用于直接存储数据。通常定义为 private（私有），避免外部直接访问和修改，防止数据被随意篡改（封装原则）。

```
namespace C_test
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Student student = new Student();
            student.age = 40;
            student.name = "Test";
            Student student2 = new Student();
            student2.age = 80;
            student2.name = "Test";
        }
    }
    class Student
    {
        public int age;           //实例字段
        public string name="default"; //给实例字段设置默认值
        public static int Amount; //静态字段
        public Student() //实例构造器（给实例字段用的构造器）
        {
        }
        static Student() //静态构造器（给静态字段用的，会在数据加载的时候执行而且只会执行一次）
        {
        }
    }
}
```


属性

• 什么是属性

- 属性（property）是一种用于访问对象或类型的特征的成员，特征反映了状态
- 属性是字段的自然扩展
 - 从命名上看，field更偏向于实例对象在内存中的布局，property更偏向于反映现实世界对象的特征
 - 对外：暴露数据，数据可以是存储在字段里的，也可以是动态计算出来的
 - 对内：保护字段不被非法值“污染”
- 属性由Get/Set方法对进化而来
- 又一个“语法糖”——属性背后的秘密

• 属性的声明

- 完整声明——后台（back）成员变量与访问器（注意使用code snippet和refactor工具）
- 简略声明——只有访问器（查看IL代码）
- 动态计算值的属性
- 注意实例属性和静态属性
- 属性的名字一定是名词
- 只读属性——只有getter没有setter
 - 尽管语法上正确，几乎没有人使用“只写属性”，因为属性的主要目的是通过向外暴露数据而表示对象/类型的状态

• 属性与字段的关系

- 一般情况下，它们都用于表示实体（对象或类型）的状态
- 属性大多数情况下是字段的包装器（wrapper）
- 建议：永远使用属性（而不是字段）来暴露数据，即字段永远都是private或protected的

属性是包裹字段的“访问器”，提供受控的访问方式（读 / 写），是字段的“对外接口”，包含 get（读取值）和 set（设置值）方法，可以在读写时添加逻辑（比如数据验证、日志记录）。

```
class Student
{
    private int age;           //实例字段
    public int GetAge()        //get属性获取值
    {
        return age;
    }
    public void SetAge(int Value) //set属性设置值
    {
        if(Value>=0 && Value < 120) //在属性内可以添加逻辑
        {
            this.age = Value;
        }
        else
        {
            throw new Exception("年龄输入错误!");
        }
    }
}
```

这样设置的话主函数访问就需要这么写：

```
Student student = new Student();
student.SetAge(20);
```

其实还有一个更好的方法，可以使用微软自己设置的set和get函数：

```
private int age;           //私有字段
public int Age //公有属性
{
    get { return this.age; }
    set //set这没有输入的参数因为微软默认value是
```

```

    {
        if (value >= 0 && value <= 120) //value是局部关键字，用来接收输入的关键字，放外面就不是关键字了
        {
            this.age = value;
        }
        else
        {
            throw new Exception("年龄错误!");
        }
    }
}
}

```

这样又可以像之前那样打点访问了：

```

Student student = new Student();
student.Age = 20;

```

十二、索引器 (C++叫迭代器) 和常量

索引器（概述）

- 什么是索引器
 - 索引器 (indexer) 是这样一种成员：它使对象能够用与数组相同的方式（即使用下标）进行索引
- 索引器的声明
 - 参见C#语言定义文档
 - 注意：没有静态索引器

索引器 (Indexer) 是 C# 中一种特殊的类成员，允许你像访问数组一样访问对象的内部数据，本质是给类提供“数组式”的访问语法。可以把它理解为：给类定制一个“[] 运算符”，让对象能通过 对象名[索引] 的方式读取 / 修改内部数据，而不用显式调用方法（比如 GetData(int index)/SetData(int index, value)）。

```

using System;
// 自定义一个简单的字符串容器类
class StringContainer
{
    // 内部存储数据的数组
    private string[] _strings = new string[3] { "张三", "李四", "王五" };
    // 定义索引器（核心）
    // 语法: public 返回值类型 this[参数类型 索引名] { get; set; }
    public string this[int index]
    {
        // 读取索引对应的值（get访问器）
        get
        {
            // 简单的边界检查（避免数组越界）
            if (index < 0 || index >= _strings.Length)
            {
                throw new ArgumentOutOfRangeException("index", "索引超出范围");
            }
            return _strings[index];
        }
        // 修改索引对应的值（set访问器）
    }
}

```

```

        set
        {
            if (index < 0 || index >= _strings.Length)
            {
                throw new ArgumentOutOfRangeException("index", "索引超出范围");
            }
            // set访问器中, value是关键字, 代表外部传入的赋值内容
            _strings[index] = value;
        }
    }
}
// 可选: 获取内部数组长度, 方便外部遍历
public int Length => _strings.Length;
}
// 测试索引器
class Program
{
    static void Main()
    {
        // 创建对象
        StringContainer container = new StringContainer();
        // 1. 读取索引器的值 (像数组一样用[])
        Console.WriteLine("读取索引0的值: " + container[0]); // 输出: 张三
        Console.WriteLine("读取索引1的值: " + container[1]); // 输出: 李四
        // 2. 修改索引器的值
        container[1] = "赵六";
        Console.WriteLine("修改后索引1的值: " + container[1]); // 输出: 赵六
        // 3. 遍历 (结合Length)
        Console.WriteLine("\n遍历所有元素: ");
        for (int i = 0; i < container.Length; i++)
        {
            Console.WriteLine(container[i]);
        }
    }
}

```

常量

- 什么是常量
 - 常量 (constant) 是表示常量值 (即, 可以在编译时计算的值) 的类成员
 - 常量隶属于类型而不是对象, 即没有“实例常量”
 - “实例常量”的角色由只读实例字段来担当
 - 注意区分成员常量与局部常量
- 常量的声明
- 各种“只读”的应用场景
 - 为了提高程序可读性和执行效率——常量
 - 为了防止对象的值被改变——只读字段
 - 向外暴露不允许修改的数据——只读属性 (静态或非静态), 功能与常量有一些重叠
 - 当希望成为常量的值其类型不能被常量声明接受时 (类/自定义结构体) ——静态只读字段

例如Math.PI就是一个常量, 以下就是常量PI的定义, 常量的运行效率要比变量更高。常量在定义的时候需要赋值, 赋值后不能再修改。常量又可以分为成员常量 (类的成员) 和局部常量 (参与组成方法体的算法)。

```
IndexerExample - Microsoft Visual Studio Express 2013 for Windows Desktop
FILE EDIT VIEW PROJECT BUILD DEBUG TEAM TOOLS TEST WINDOW HELP
Start < Debug Any CPU
Program.cs
System.Int32
Assembly mscorlib.dll, v4.0.0.0
4
5 using System.Globalization;
6 using System.Runtime;
7 using System.Runtime.InteropServices;
8 using System.Security;
9
10 namespace System
11 {
12     public struct Int32 : IComparable, IFormattable, IConvertible, IComparable<int>, IEquatable<int>
13     {
14         public const int MaxValue = 2147483647;
15         public const int MinValue = -2147483648;
16
17         public int CompareTo(int value);
18         public int CompareTo(object value);
19         public bool Equals(int obj);
20         public override bool Equals(object obj);
21         public override int GetHashCode();
22         public TypeCode GetTypeCode();
23         public static int Parse(string s);
24         public static int Parse(string s, IFormatProvider provider);
25         public static int Parse(string s, NumberStyles style);
26         public static int Parse(string s, NumberStyles style, IFormatProvider provider);
27         public override string ToString();
28         public string ToString(IFormatProvider provider);
29         public string ToString(string format);
30         public string ToString(string format, IFormatProvider provider);
31         public static bool TryParse(string s, out int result);
32     }
33 }
```

常量编译时就决定了，而静态只读变量初次运行被调用才决定，所以常量的运行效率要比静态只读变量要高。

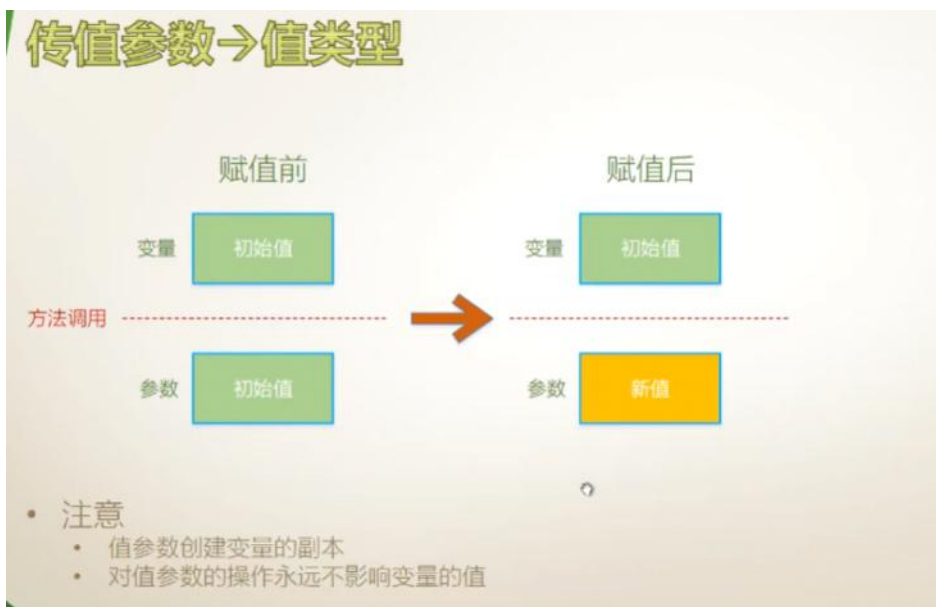
十三、参数

1、传值参数—值参数

场景 1：值参数 + 值类型（最基础）

值类型变量：存储的是实际数据（比如int num = 10，变量直接存 10）。

值参数传递：复制一份实际数据给方法，方法内修改副本，原变量完全不受影响。



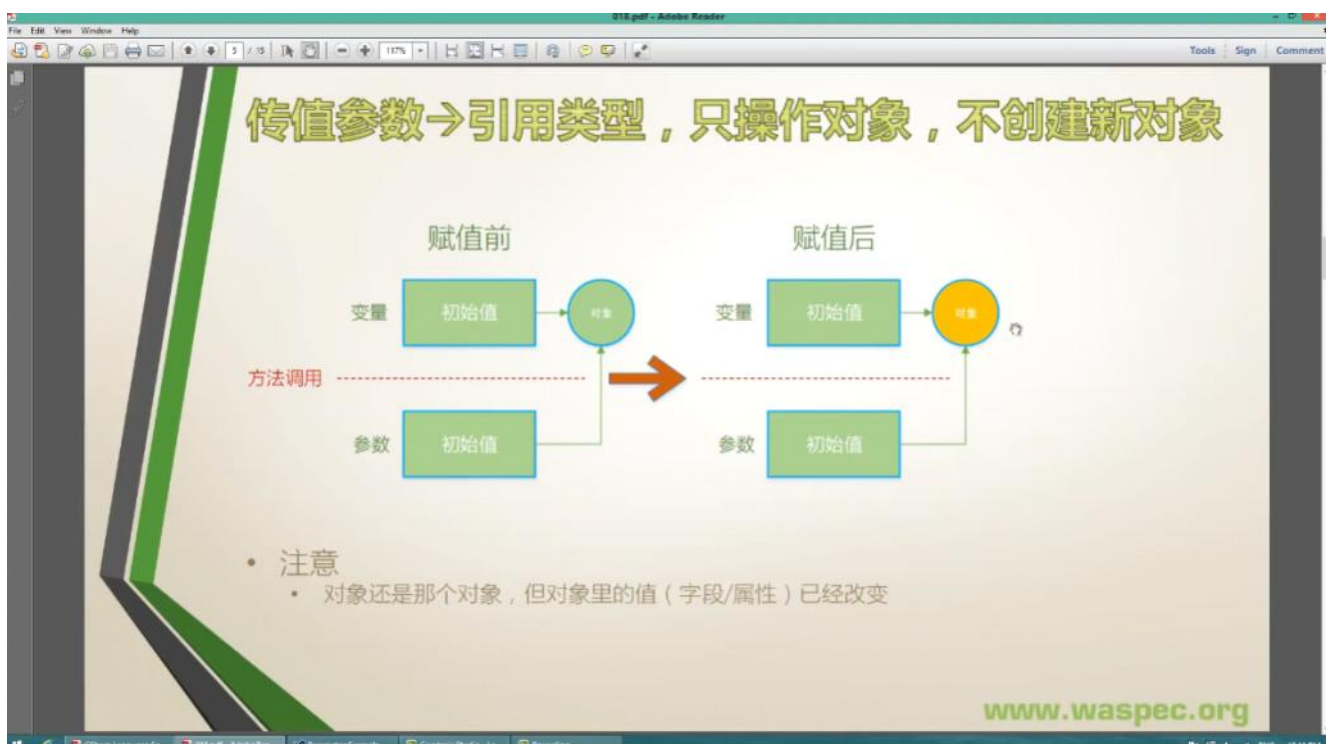
```
csharp ^
using System;

class Program
{
    // 值参数 + 值类型
    static void ModifyValue(int num) // num是值参数，int是值类型
    {
        num = 20; // 修改的是“10”这个数据的副本
        Console.WriteLine("方法内num: " + num); // 输出20
    }

    static void Main()
    {
        int original = 10; // 值类型变量，存的是实际数据10
        ModifyValue(original);
        Console.WriteLine("方法外original: " + original); // 输出10（原数据不变）
    }
}
```

场景 2：值参数 + 引用类型

引用类型变量：存储的是“对象在堆上的内存地址”（比如Person p = new Person()，变量 p 存的是对象的地址，不是对象本身）。
值参数传递：复制的是“地址的副本”——方法内通过副本地址修改对象的属性，会影响原对象；但修改“副本地址本身”（比如重新 new），不会影响原变量的地址。




```
csharp ^
// 自定义引用类型（class是引用类型）
class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}

class Program
{
    // 值参数 + 引用类型
    static void ModifyPerson(Person p) // p是值参数，Person是引用类型
    {
        // 1. 修改对象的属性（通过地址副本操作堆上的对象）→ 影响原对象
        p.Age = 30;
        Console.WriteLine("方法内Age: " + p.Age); // 输出30

        // 2. 修改地址副本本身（重新new）→ 与原变量无关
        p = new Person { Name = "李四", Age = 25 };
        Console.WriteLine("方法内新对象Age: " + p.Age); // 输出25
    }

    static void Main()
    {
        Person original = new Person { Name = "张三", Age = 20 }; // 引用类型变量，存对象地址
        ModifyPerson(original);

        // 原对象的属性被修改了（因为操作的是同一个堆对象）
        Console.WriteLine("方法外Age: " + original.Age); // 输出30
        // 原变量的地址没变化（还是指向原来的对象）
        Console.WriteLine("方法外Name: " + original.Name); // 输出张三
    }
}
```

2、传地址—引用参数

场景 3: ref 参数 + 值类型

ref 传递: 直接传递值类型变量的“内存地址”，方法内修改的是原变量的实际数据，会同步影响原变量。



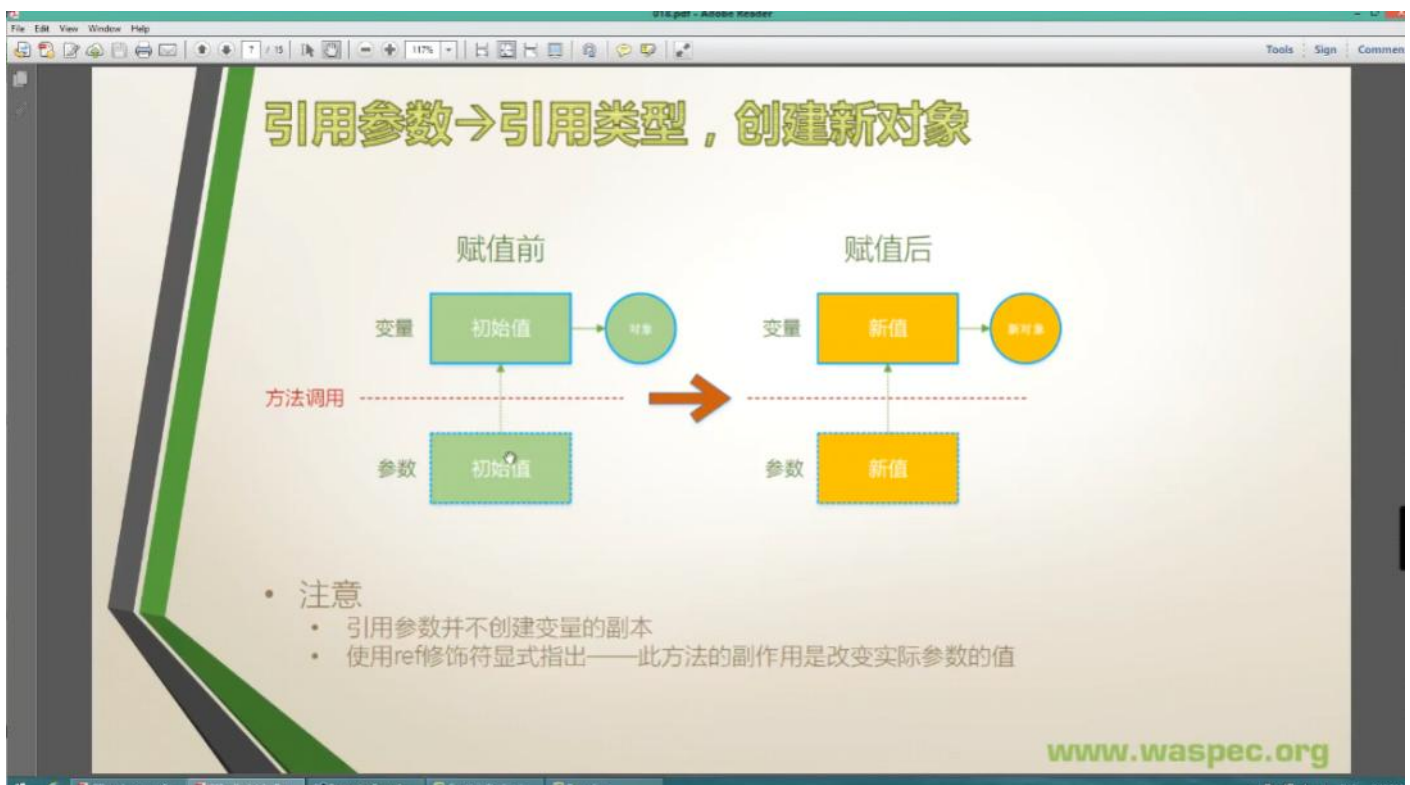
```
csharp ^
using System;

class Program
{
    // ref参数 + 值类型
    static void ModifyValueRef(ref int num) // ref修饰，int是值类型
    {
        num = 20; // 直接修改原变量的内存地址里的数据
        Console.WriteLine("方法内num: " + num); // 输出20
    }

    static void Main()
    {
        int original = 10; // 值类型变量
        ModifyValueRef(ref original); // 必须加ref
        Console.WriteLine("方法外original: " + original); // 输出20 (原数据被修改)
    }
}
```

场景 4: ref 参数 + 引用类型

ref 传递: 直接传递引用类型变量的“地址本身”（不是地址副本）—— 方法内无论是修改对象属性，还是重新 new 对象（修改地址），都会影响原变量。



```

csharp ^
class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}

class Program
{
    // ref参数 + 引用类型
    static void ModifyPersonRef(ref Person p) // ref修饰，Person是引用类型
    {
        // 1. 修改对象属性 → 影响原对象（和值参数+引用类型一样）
        p.Age = 30;
        Console.WriteLine("方法内Age: " + p.Age); // 输出30

        // 2. 修改地址本身（重新new）→ 原变量也会指向新对象（这是和值参数的核心区别）
        p = new Person { Name = "李四", Age = 25 };
        Console.WriteLine("方法内新对象Age: " + p.Age); // 输出25
    }

    static void Main()
    {
        Person original = new Person { Name = "张三", Age = 20 };
        ModifyPersonRef(ref original); // 必须加ref

        // 原变量的地址被修改，指向了新对象！
        Console.WriteLine("方法外Age: " + original.Age); // 输出25
        Console.WriteLine("方法外Name: " + original.Name); // 输出李四
    }
}

```

3. 输出参数

需要显式加 `out` 修饰符，专门用于方法返回多个值（弥补 C# 方法只能返回一个值的限制）。

原理：和 `ref` 类似（传递地址），但要求更严格：

实参可以不初始化（方法内必须给 `out` 参数赋值）；

方法内部必须为 `out` 参数赋值，否则编译报错。

场景：比如一个方法既要返回计算结果，又要返回是否成功。

10.6.1.3 输出形参

用 `out` 修饰符声明的形参是输出形参。类似于引用形参，**输出形参不创建新的存储位置**。相反，输出形参表示的存储位置恰是在该方法调用中作为实参给出的那个变量所表示的存储位置。

当形参为输出形参时，方法调用中的相应实参必须由关键字 `out` 并后接一个与形参类型相同的 *variable-reference*（第 5.3.3 节）组成。**变量在可以作为输出形参传递之前不一定需要明确赋值**，但是在将变量作为输出形参传递的调用之后，该变量被认为是明确赋值的。

在方法内部，与局部变量相同，输出形参最初被认为是未赋值的，因而必须在使用它的值之前明确赋值。

在方法返回之前，该方法的每个输出形参都必须明确赋值。

声明为分部方法（第 10.2.7 节）或迭代器（第 10.14 节）的方法不能有输出形参。

输出形参通常用在需要产生多个返回值的方法中。例如：

```

using System;
class Test
{
    static void SplitPath(string path, out string dir, out string name) {
        int i = path.Length;
        while (i > 0) {
            char ch = path[i - 1];
            if (ch == '\\' || ch == '/' || ch == ':') break;
            i--;
        }
        dir = path.Substring(0, i);
        name = path.Substring(i);
    }

    static void Main() {
        string dir, name;
        SplitPath("c:\\windows\\system\\hello.txt", out dir, out name);
        Console.WriteLine(dir);
        Console.WriteLine(name);
    }
}

```

www.waspec.org

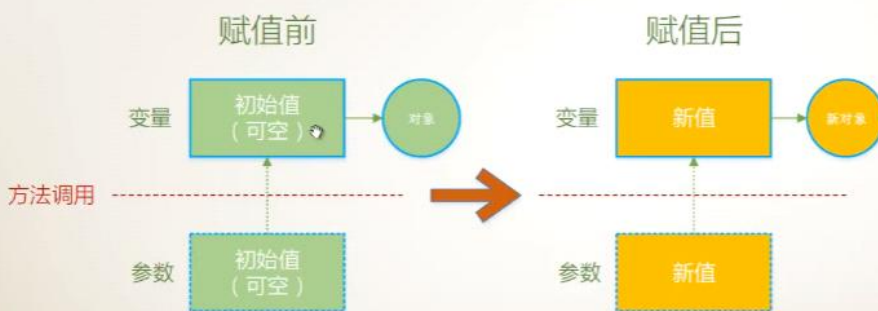
输出参数→值类型



• 注意

- 输出参数并不创建变量的副本
- 方法体内必需要有对输出变量的赋值的操作
- 使用out修饰符显式指出——此方法的副作用是通过参数向外输出值
- 从语义上来讲——ref是为了“改变”，out是为了“输出”

输出参数→引用类型



• 注意

- 输出参数并不创建变量的副本
- 方法体内必需要有对输出变量的赋值的操作
- 使用out修饰符显式指出——此方法的副作用是通过参数向外输出值
- 从语义上来讲——ref是为了“改变”，out是为了“输出”

```

csharp ^
using System;

class Program
{
    // 输出参数示例：计算两数之和，同时返回是否计算成功
    static void CalculateSum(int a, int b, out int sum, out bool isSuccess)
    {
        sum = a + b; // 必须给out参数赋值
        isSuccess = true;
    }

    static void Main()
    {
        int result; // 可以不初始化
        bool success;
        CalculateSum(5, 3, out result, out success); // 调用时加out

        Console.WriteLine("和为: " + result); // 输出: 8
        Console.WriteLine("是否成功: " + success); // 输出: True
    }
}

```

4、数组参数

4. 参数数组（params 修饰符）

需要显式加 params 修饰符，允许方法接收数量可变的同类型参数。

原理：编译器会自动把可变参数转换成数组，params 必须是方法的最后一个参数。

数组参数

- 必需是形参列表中的最后一个，由params修饰
- 举例：String.Format方法和String.Split方法

```

string str = "Tim;Tom;Amy;Lisa";
string[] strings = str.Split(';',' ',';',' '); //str.Split表示分离
foreach (var name in strings) {
    Console.WriteLine(name);
}

```



```
csharp ^ 运行 文件 窗口 设置 帮助

using System;

class Program
{
    // 参数数组示例：计算任意多个整数的和
    static int Sum(params int[] numbers)
    {
        int total = 0;
        foreach (int num in numbers)
        {
            total += num;
        }
        return total;
    }

    static void Main()
    {
        // 可以传任意数量的参数（包括0个）
        int sum1 = Sum(1, 2, 3); // 传3个参数
        int sum2 = Sum(10, 20, 30, 40); // 传4个参数
        int sum3 = Sum(); // 传0个参数

        Console.WriteLine("sum1: " + sum1); // 输出: 6
        Console.WriteLine("sum2: " + sum2); // 输出: 100
        Console.WriteLine("sum3: " + sum3); // 输出: 0
    }
}
```

5、具名参数

具名参数

- 参数的位置不再受约束

6、可选参数

可选参数

- 参数因为具有默认值而变得“可选”
- 不推荐使用可选参数

7、扩展方法this参数

扩展方法（this参数）

- 方法必需是公有、静态的，即被public static所修饰
- 必需是形参列表中的第一个，由this修饰
- 必需由一个静态类（一般类名为SomeTypeExtension）来统一收纳对SomeType类型的扩展方法
- 举例：LINQ方法

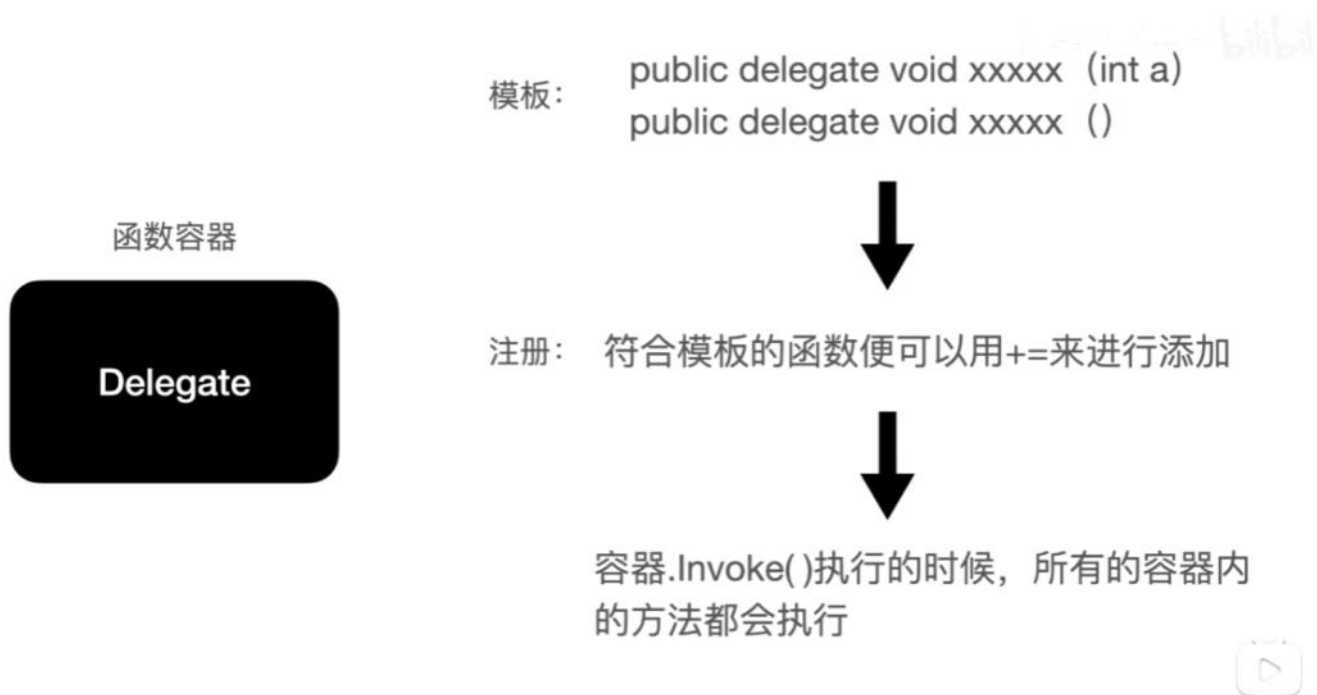
Extension Methods

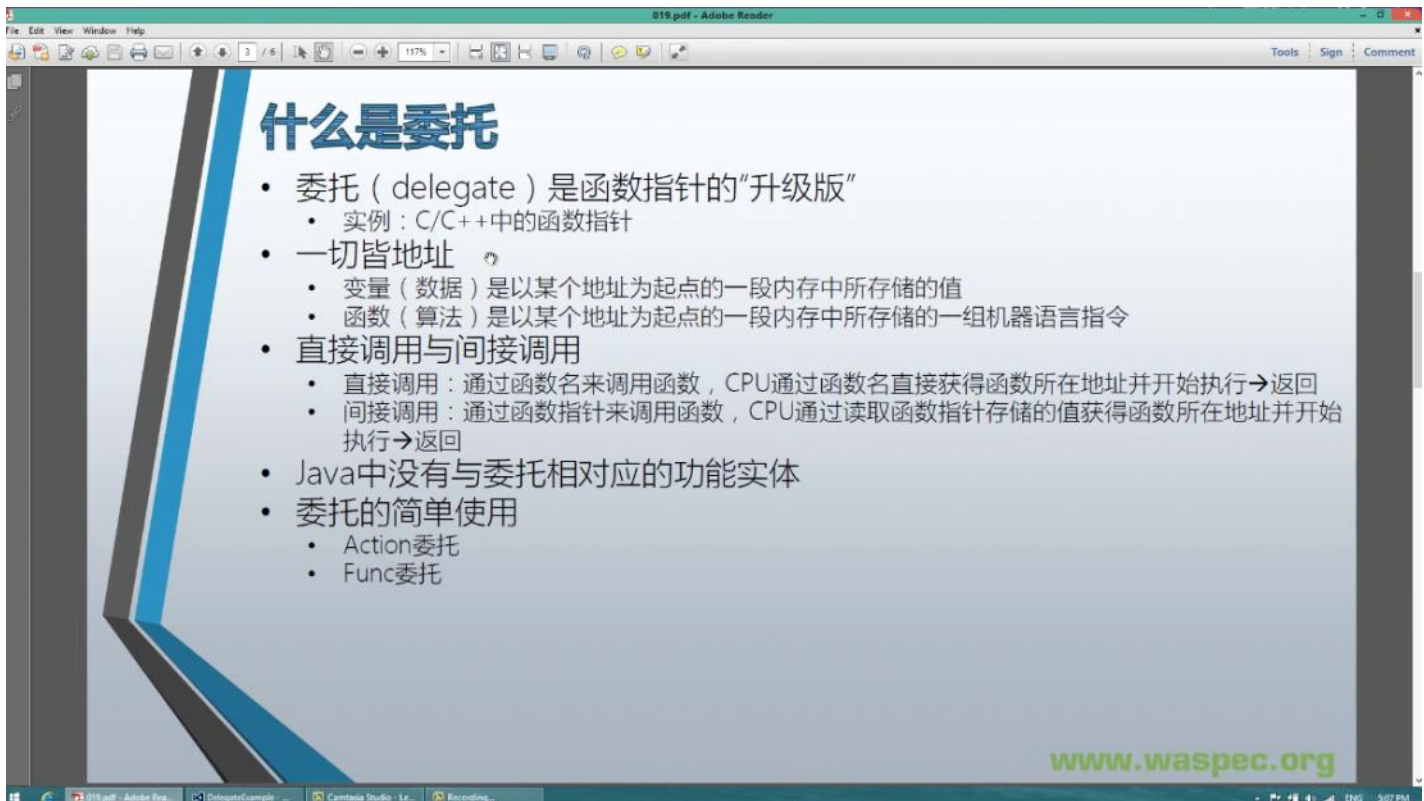
各种参数的使用场景总结

- 传值参数：参数的默认传递方式
- 输出参数：用于除返回值外还需要输出的场景
- 引用参数：用于需要修改实际参数值的场景
- 数组参数：用于简化方法的调用
- 具名参数：提高可读性
- 可选参数：参数拥有默认值
- 扩展方法（this参数）：为目标数据类型“追加”方法

十四：委托

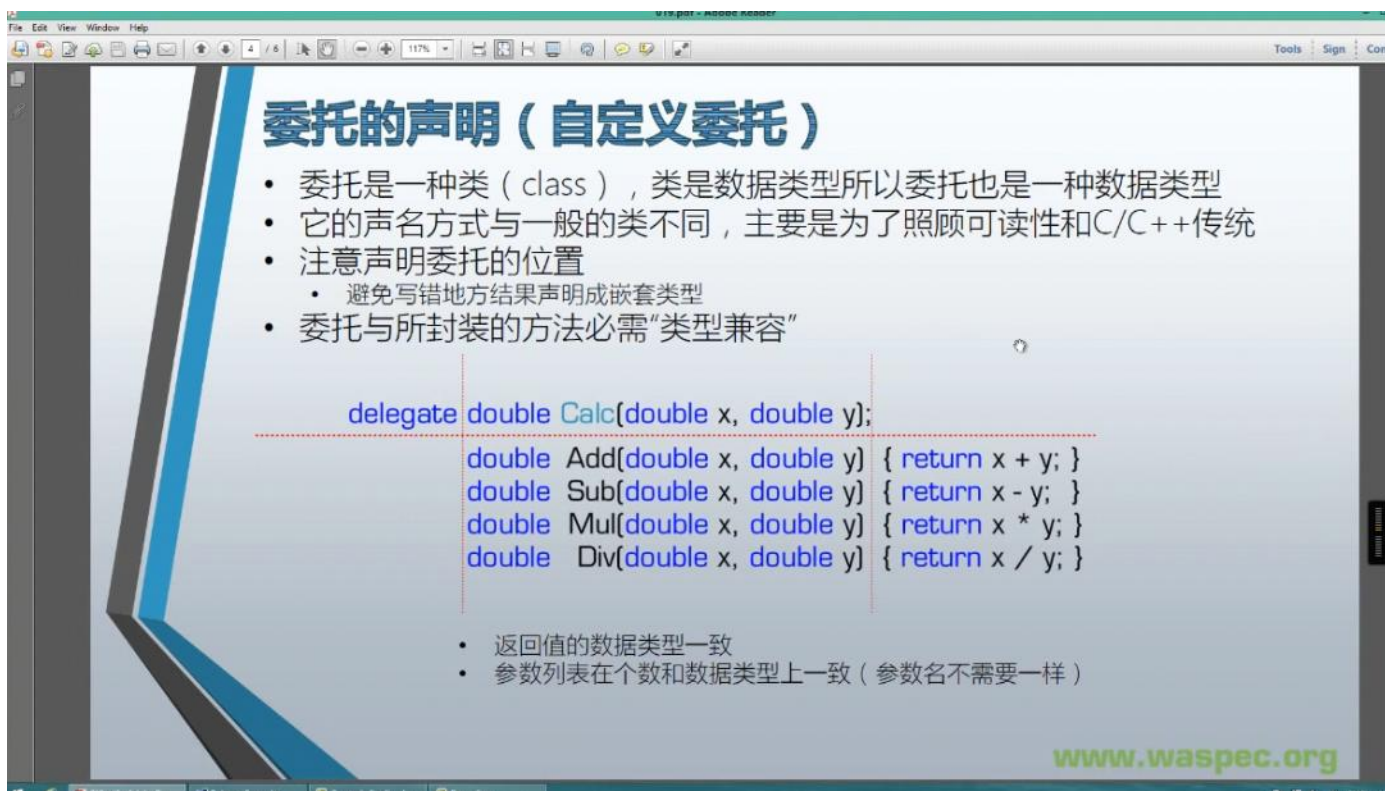
委托可以比喻成一个装方法的箱子，箱子里装了很多个方法，当需要一批方法的时候，就调用这个委托就行了。





```
using System;
class Program
{
    static void Main()
    {
        Calculator calculator = new Calculator();
        Action action = new Action(calculator.report);
        calculator.report(); //直接调用
        action.Invoke();     //使用委托间接调用
        action();            //简化版的间接调用
        Func<int, int, int> func = new Func<int, int, int>(calculator.Add); //使用Func委托间接调用
        Func<int, int, int> func1 = new Func<int, int, int>(calculator.Sub);
    }
}
class Calculator
{
    public void report()
    {
        Console.WriteLine("I have 3 methods");
    }
    public int Add(int a, int b)
    {
        int result = a + b;
        return result;
    }
    public int Sub(int a, int b)
    {
        return a - b;
    }
}
```

以上的委托都是C#自带的委托，以下是自定义委托的声明：



由此可见，声明的委托的返回值类型和参数类型必须和方法的返回值类型和参数类型相一致，参数名不用一致也行。

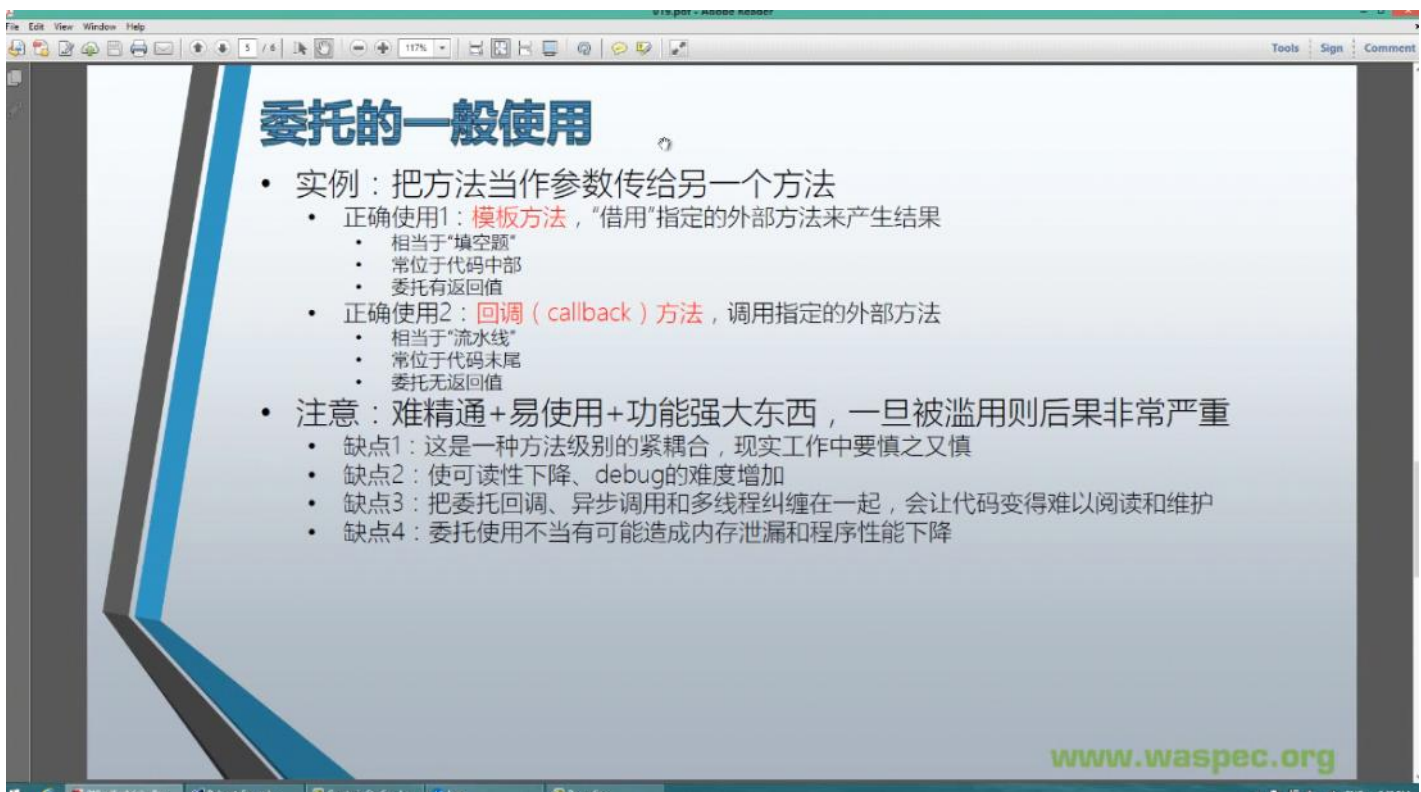
```
using System;
using System.Transactions;
class Program
{
    static void Main()
    {
        Calculator calculator = new Calculator(); //使用委托来封装方法
        Calc calc1 = new Calc(calculator.Add);
        Calc calc2 = new Calc(calculator.Sub);
        Calc calc3 = new Calc(calculator.Multiply);
        Calc calc4 = new Calc(calculator.Divide);
        double a = 100;
        double b = 200;
        double c = 0;
        c = calc1.Invoke(a, b); //使用委托间接调用
        Console.WriteLine(c);
        c = calc2.Invoke(a, b);
        Console.WriteLine(c);
        c = calc3(a, b); //间接调用（仿照函数指针来调用）
        Console.WriteLine(c);
        c = calc4(a, b);
        Console.WriteLine(c);
    }
}

public delegate double Calc(double x, double y); //委托是类，因此与类同级
class Calculator
{
    public double Add(double x, double y)
    {
        return x + y;
    }
    public double Sub(double x, double y)
```

```

{
    return x - y;
}
public double Multiply(double x, double y)
{
    return x * y;
}
public double Divide(double x, double y)
{
    return x / y;
}
}

```



委托的一般使用

- 实例：把方法当作参数传给另一个方法
 - 正确使用1：模板方法，“借用”指定的外部方法来产生结果
 - 相当于“填空题”
 - 常位于代码中部
 - 委托有返回值
 - 正确使用2：回调（callback）方法，调用指定的外部方法
 - 相当于“流水线”
 - 常位于代码末尾
 - 委托无返回值
- 注意：难精通+易使用+功能强大东西，一旦被滥用则后果非常严重
 - 缺点1：这是一种方法级别的紧耦合，现实工作中要慎之又慎
 - 缺点2：使可读性下降、debug的难度增加
 - 缺点3：把委托回调、异步调用和多线程纠缠在一起，会让代码变得难以阅读和维护
 - 缺点4：委托使用不当有可能造成内存泄漏和程序性能下降

www.waspec.org

```

csharp ^
using System;

// 1. 定义委托：规定“要传的方法”的签名（有返回值，两个int参数）
public delegate int CalcDelegate(int a, int b);

class Program
{
    // 2. 模板方法：接收一个委托作为“填空”，用它来计算
    static int Compute(int x, int y, CalcDelegate calc)
    {
        Console.WriteLine("模板方法开始执行...");
        // “借用”外部传入的方法来产生结果
        int result = calc(x, y);
        Console.WriteLine("模板方法执行结束，结果是：" + result);
        return result;
    }

    // 3. 两个具体的“填空”方法
    static int Add(int a, int b) => a + b;
    static int Multiply(int a, int b) => a * b;

    static void Main()
    {
        // 4. 把方法当参数传给模板方法
        Compute(2, 3, Add); // 输出：模板方法...结果是5
        Compute(2, 3, Multiply); // 输出：模板方法...结果是6
    }
}

```


Compute 是**模板**，只负责流程，不关心具体怎么算。
Add/Multiply 是你填的**答案**，由委托传进去。
就像考试填空题：题目（模板）固定，答案（方法）你自己写。

```
csharp ^ 运行 文件 窗口 帮助

using System;

// 1. 定义委托：无返回值，接收一个字符串消息
public delegate void CallbackDelegate(string message);

class Program
{
    // 2. 核心业务方法：下载文件，完成后调用回调
    static void DownloadFile(string fileName, CallbackDelegate callback)
    {
        Console.WriteLine($"开始下载: {fileName}...");
        // 模拟下载耗时
        System.Threading.Thread.Sleep(1000);
        Console.WriteLine("下载完成!");

        // 流水线末尾：调用外部传入的回调方法
        callback($"文件 {fileName} 已保存到本地");
    }

    // 3. 两个具体的回调方法
    static void ShowMessageBox(string msg) => Console.WriteLine($"【弹窗提示】{msg}");
    static void WriteLog(string msg) => Console.WriteLine($"【日志记录】{msg}");

    static void Main()
    {
        // 4. 把回调方法当参数传进去
        DownloadFile("demo.pdf", ShowMessageBox);
        DownloadFile("data.zip", WriteLog);
    }
}
```

DownloadFile 是**流水线**，负责下载流程。
下载完成后，在末尾 “调用” 你传的回调方法，通知结果。
就像快递：快递员（核心方法）负责送货，你（回调）决定收到后是发短信还是发邮件。

委托的高级使用

- 多播 (multicast) 委托
- 隐式异步调用
 - 同步与异步的简介
 - 中英文的语言差异
 - 同步：你做完了我（在你的基础上）接着做
 - 异步：咱们两个同时做（相当于汉语中的“同步进行”）
 - 同步调用与异步调用的对比
 - 每一个运行的程序是一个进程 (process)
 - 每个进程可以有一个或者多个线程 (thread)
 - 同步调用是在同一线程内
 - 异步调用的底层机理是**多线程**
 - 串行==同步==单线程，并行==异步==多线程
 - 隐式多线程 v.s. 显式多线程
 - 直接同步调用：使用方法名
 - 间接同步调用：使用单播/多播委托的Invoke方法
 - 隐式异步调用：使用委托的BeginInvoke
 - 显式异步调用：使用Thread或Task
- 应该适时地使用接口 (interface) 取代一些对委托的使用
 - Java完全地使用接口取代了委托的功能，即Java没有与C#中委托相对应的功能实体

www.waspec.org

多播委托 (Multicast)：
多播委托 = 一个委托变量绑定多个方法

调用时会按绑定顺序依次执行

```
using System;
// 1. 定义一个无返回值的委托（多播委托常用无返回值）
public delegate void NotifyDelegate();
class Program
{
    static void Main()
    {
        // 2. 分别定义几个要执行的方法
        void SendEmail() => Console.WriteLine("发送邮件通知");
        void SendSMS() => Console.WriteLine("发送短信通知");
        void ShowPopup() => Console.WriteLine("弹出窗口提示");
// 3. 多播委托：用 += 把多个方法“绑”到一起
        NotifyDelegate notify = null; // 先初始化为空
        notify += SendEmail; // 绑定第一个方法
        notify += SendSMS; // 绑定第二个方法
        notify += ShowPopup; // 绑定第三个方法
// 4. 调用一次委托，三个方法按顺序执行
        Console.WriteLine("=== 触发多播委托 ===");
        notify();
// 5. 可以用 -= 解绑某个方法
        notify -= SendSMS;
        Console.WriteLine("\n=== 解绑短信后再次触发 ===");
        notify(); // 只执行邮件和弹窗
    }
}
```

隐式异步调用：用委托做“后台干活”

同步：你做完我再做（排队）

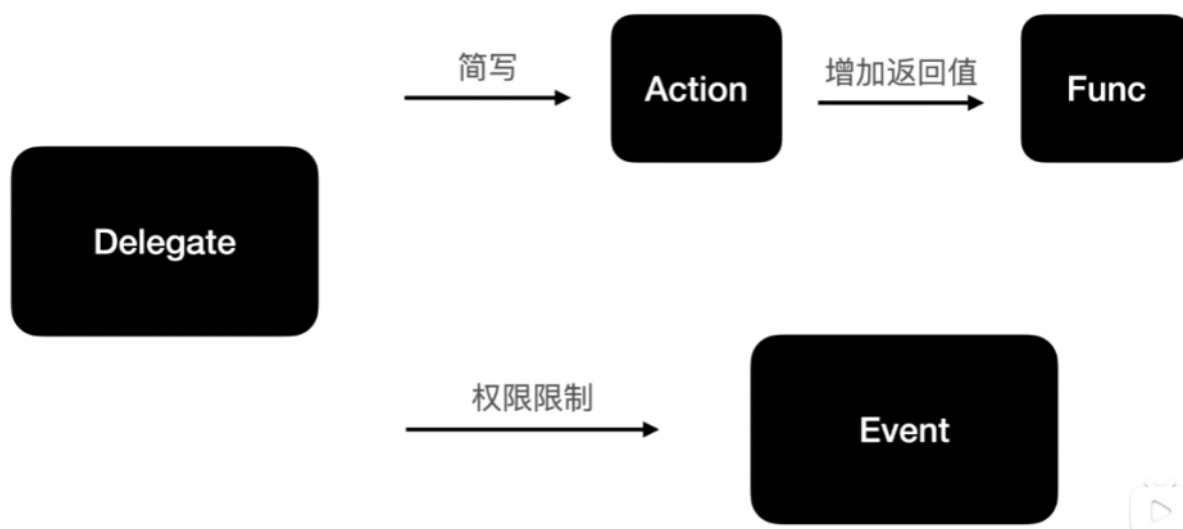
异步：咱们同时做（并行）

```
using System;
using System.Threading;
// 1. 定义一个有返回值的委托
public delegate int CalcDelegate(int a, int b);
class Program
{
    static void Main()
    {
        // 2. 定义一个耗时的计算方法
        int SlowAdd(int a, int b)
        {
            Console.WriteLine($"[$] 开始慢计算...（模拟耗时3秒）");
            Thread.Sleep(3000); // 模拟3秒耗时
            Console.WriteLine($"[$] 慢计算完成！");
            return a + b;
        }
        CalcDelegate calc = SlowAdd;
// 3. 同步调用（会卡住主线程）
        Console.WriteLine("=== 同步调用 ===");
        int syncResult = calc(1, 2);
        Console.WriteLine("同步结果：" + syncResult);
        Console.WriteLine("同步调用结束，主线程被卡住了\n");
// 4. 隐式异步调用（BeginInvoke，不卡住主线程）
        Console.WriteLine("=== 异步调用 ===");
        var asyncResult = calc.BeginInvoke(3, 4, null, null); // 后台开始算
        Console.WriteLine("[*] 主线程继续做别的事...");
    }
}
```

```
// 模拟主线程在干活
    for (int i = 0; i < 3; i++)
    {
        Console.WriteLine($"[*] 主线程在执行第 {i + 1} 步...");
        Thread.Sleep(1000);
    }
// 5. 等待异步完成并获取结果
    int asyncResultValue = calc.EndInvoke(asyncResult);
    Console.WriteLine("异步结果: " + asyncResultValue);
    Console.WriteLine("异步调用结束");
}
}
```

十五：事件

委托可以比喻成一个装方法的箱子，箱子里装了很多个方法，当需要一批方法的时候，就调用这个委托就行了。



但是委托存在一种风险就是它可以被直接赋值，就会导致其容器内的函数存在丢失的风险，假设我们delegate1容器内存在着数个函数而delegate2没有，用等号赋值完成后委托1的注册函数就会被清空（delegate1的地址指向了delegate2的地址）。



而事件就解决了这种问题，事件的本质就是特殊的委托，赋值的权限设置成了private，比委托更加安全。写法上就是多加了个event关键词

函数容器

Event

本质也是Delegate，但是赋值的权限设置成了Private

```
public delegate void xxxxx ()  
public event xxxxx delegate1 ()
```

C#封装好了Action和Function两种委托，最多支持16个参数的模版，其中Action不带返回值而Function带返回值（返回值就是最后一个参数的类型），写法如下：

函数容器

Action

```
public Action xxxxx ()  
public Action<int> xxxxx ()  
public Action<bool,int.....*16> xxxxx ()
```

函数容器

Function

```
public Func<int> xxxxx () return int  
public Func<int,bool> xxxxx () return bool
```

但是若多个返回值发生，则最后保留的是最后注册函数的返回值

十六、类

1、构造器和析构器

```

1 using System;
2
3 namespace HelloClass {
4     class Program {
5         static void Main(string[] args) {
6             Student stu = new Student(1, "Timothy");
7             stu.Report();
8         }
9     }
10 }
11
12 class Student {
13     public Student(int id, string name) {
14         this.ID = id;
15         this.Name = name;
16     }
17
18     ~Student() { 析构器，用来释放内存，防止内存泄漏
19         Console.WriteLine("Bye bye! Release the system resources ...");
20     }
21 }

```

构造器

2、类的继承

• 类的继承

- 类在功能上的扩展 (extend)
- 只能有一个基类，但可以实现多个其接口
- 类访问级别对继承的影响
- sealed类不能被继承

• 成员的继承与访问

- 派生类对继承成员的访问
- 派生类对基类成员的访问
- 构造器的不可继承性

• 面向对象的实现风格

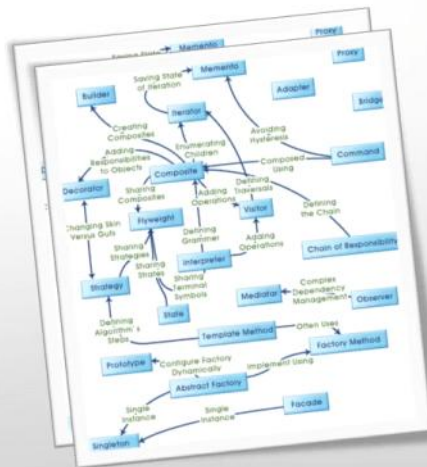
- Class-based
- Prototype-based

10.1 Class declarations

A class-declaration is a type-declaration (§9.6) that declares a new class.

class-declaration:

attributes_{opt} class-modifiers_{opt} partial_{opt} class identifier type-parameter-list_{opt}
class-base_{opt} type-parameter-constraints-clauses_{opt} class-body ;_{opt}



一般来说，基类和派生类是一对，父类和子类是一对。

```

using System;
using System.Transactions;
class Program
{
    static void Main()
    {
        Type t = typeof(Car); //t表示Car类
        Type tb = t.BaseType; //Car类的基类是Vehicle
        Console.WriteLine(tb.FullName); //输出的是Vehicle
        Type tc = tb.BaseType; //输出的是System.Object类
        Console.WriteLine(Car is Object); //输出的是true
        Object o1 = new Vehicle(); //父类类型变量可以引用子类类型的实例（多态）
        Object o2 = new Car();
    }
}

```

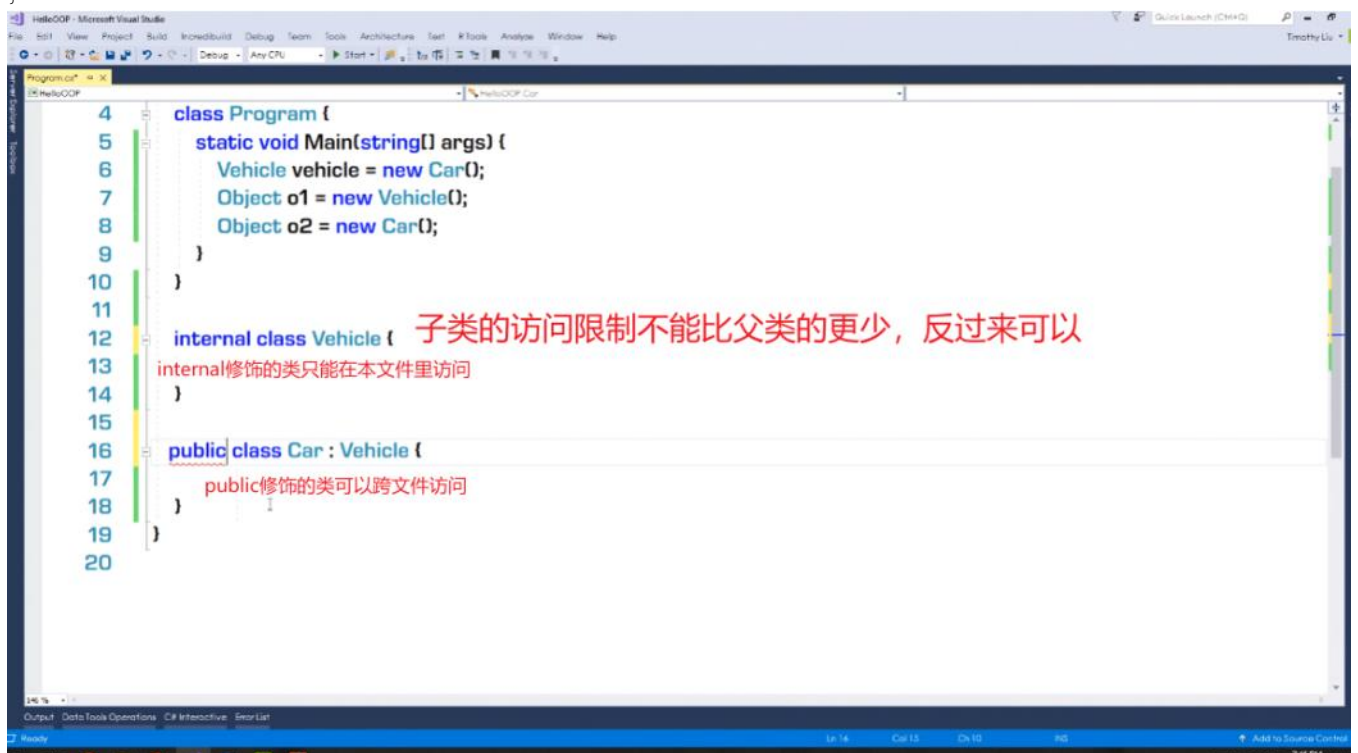


```

}
sealed class Test //sealed修饰的类不能再派生类（断子绝孙）
{
}
class Vehicle: Object //Object类是所有继承类的源头
{
}
class Car : Vehicle //Car类从Vehicle类派生而来，Car继承了Vehicle类
{
}

class Toy : Vehicle //一个子类只能有一个父类，一个父类可以有多个子类（孩子不能有多爹，爸爸可以有多个孩子）
{
}

```



子类对父类的继承是全盘继承的，在派生与继承的过程中进行的是扩展，类成员只可能是越来越多。不要贸然引入新的类成员，不然后期很难去掉了。横向扩展是对类成员个数的扩充，纵向成员是对类版本的更新或者重写。

派生类对继承成员的访问，父类的成员访问修饰符决定了子类能不能访问：

public: 任何地方都能访问

protected: 子类和同一个包内可以访问

private: 子类完全不能访问 ✖

```
csharp ^
public class Animal
{
    public string Name { get; set; } // 子类可直接访问
    protected int Age { get; set; } // 子类可直接访问
    private string _id { get; set; } // 子类不能访问
}

public class Dog : Animal
{
    public void ShowInfo()
    {
        Console.WriteLine(Name); // ✓ 继承的public成员
        Console.WriteLine(Age); // ✓ 继承的protected成员
        // Console.WriteLine(_id); // ✗ 父类private成员，子类访问不了
    }
}
```

构造器不可继承

父类的构造方法，子类不能直接继承，必须通过base()显式调用。

```
csharp ^
public class Animal
{
    public Animal(string name)
    {
        Name = name;
    }
    public string Name { get; set; }
}

public class Cat : Animal
{
    // 必须调用父类构造器，否则编译报错
    public Cat(string name) : base(name) {}
}
```

```
csharp ^
Cat cat = new Cat("咪咪"); // 会先调用Animal的构造器，再执行Cat的构造器
Console.WriteLine(cat.Name); // 输出：咪咪
```

面向对象的实现风格

(1) Class-based (基于类)

主流语言 (Java/C#/C++) 都是这种：先定义类，再用类创建实例对象。

例子：上面的Animal和Bird就是典型的基于类的实现。

(2) Prototype-based (基于原型)

JavaScript 是这种：没有类的概念，直接用对象作为原型，新对象克隆原型并扩展。

总结一下：

继承就是子类复用 + 扩展父类，只能单继承，可多实现接口

sealed类是“断子绝孙”的类，不让别人继承

子类能访问父类的public/protected成员，但不能访问private

构造器不能继承，必须用base()调用父类构造器

面向对象分两种：基于类 (Java/C#) 和基于原型 (JS)

本节内容

• 类的继承

- 类成员的“横向扩展”（成员越来越多）
- 类成员的“纵向扩展”（行为改变，版本增高）
- 类成员的隐藏（不常用）
- 重写与隐藏的发生条件：函数成员，可见，签名一致

• 多态 (polymorphism)

- 基于重写机制 (virtual → override)
- 函数成员的具体行为（版本）由对象决定
- 回顾：C#语言的变量和对象都是有类型的，所以会有“代差”

横向扩展（成员越来越多）

在子类里**新增**父类没有的成员，不修改父类原有逻辑。

生活例子：父类是「手机」，子类「智能手机」在手机基础上，新增了「拍照」「上网」功能。

// 父类：手机

```
public class Phone
{
    public void Call() { Console.WriteLine("打电话"); }
}
```

// 子类：智能手机（横向扩展）

```
public class SmartPhone : Phone
{
    // 新增成员：拍照
    public void TakePhoto() { Console.WriteLine("拍照"); }
    // 新增成员：上网
    public void SurfInternet() { Console.WriteLine("上网"); }
}
```

特点：父类功能不变，子类多了新功能

纵向扩展（行为改变，版本增高）

子类**修改**父类已有成员的行为，用 virtual + override 实现。

生活例子：父类「手机」的「打电话」是按键拨号，子类「智能手机」把「打电话」改成了触屏拨号。

// 父类：手机

```
public class Phone
{
    // 标记为virtual，允许子类重写
    public virtual void Call() { Console.WriteLine("按键拨号打电话"); }
}
```

// 子类：智能手机（纵向扩展/重写）

```
public class SmartPhone : Phone
{
    // 重写父类方法，改变行为
}
```

```

        public override void Call() { Console.WriteLine("触屏拨号打电话"); }
    }

```

特点：同名方法，行为变了，版本更高。

类成员的隐藏（不常用）

子类用 `new` 关键字，把父类的同名成员 “藏起来”，不修改父类逻辑。

生活例子：父类「手机」有「Call」，子类「老人机」也定义了一个「Call」，只在老人机视角下生效。

```

public class Phone
{
    public void Call() { Console.WriteLine("通用打电话"); }
}
public class OldPhone : Phone
{
    // 隐藏父类的Call方法
    public new void Call() { Console.WriteLine("老人机大按键打电话"); }
}

```

特点：和重写不同，隐藏不遵循多态，只有用子类类型引用时才会调用子类版本。

重写与隐藏的发生条件，必须同时满足 3 个条件：

是**函数成员**（方法、属性、事件等，不是字段）

子类对父类成员**可见**（不是 `private`）

方法**签名一致**（方法名 + 参数类型 + 数量完全相同）

4、类的多态

多态 (Polymorphism) = 同一个行为，不同对象表现出不同形态。

当父类引用指向子类对象时，调用**重写**的方法，执行的是**子类的实现**，而非父类的实现

多态的 3 个必要条件（缺一不可）

继承：必须有父类和子类的继承关系；

重写：子类必须用 `override` 重写父类的 `virtual/abstract` 方法；

父类引用指向子类对象：`Parent p = new Child();`（核心！）。

```

Phone p1 = new Phone();
Phone p2 = new SmartPhone(); // 父类引用指向子类对象
p1.Call(); // 输出：按键拨号打电话（看Phone对象）
p2.Call(); // 输出：触屏拨号打电话（看SmartPhone对象）

```

在实际代码中，抽象类实现多态更为常用：

```

using System.Numerics;
// 抽象父类：无默认实现，强制子类重写
public abstract class Animal
{
    public abstract void MakeSound(); // 抽象方法，必须被重写
}
public class Dog : Animal

```

```

{
    public override void MakeSound() => Console.WriteLine("汪汪汪");
}
public class Cat : Animal
{
    public override void MakeSound() => Console.WriteLine("喵喵喵");
}
// 测试
Animal a1 = new Dog();
Animal a2 = new Cat();
a1.MakeSound(); // 汪汪汪
a2.MakeSound(); // 喵喵喵

```

5、抽象类与开闭原则

• 什么是接口和抽象类

- 接口和抽象类都是“软件工程产物”
- 具体类→抽象类→接口：越来越抽象，内部实现的东西越来越少
- 抽象类是未完全实现逻辑的类（可以有字段和非public成员，它们代表了“具体逻辑”）
- 抽象类为复用而生：专门作为基类来使用，也具有解耦功能
- 封装确定的，开放不确定的，推迟到合适的子类中去实现
- 接口是完全未实现逻辑的“类”（“纯虚类”；只有函数成员；成员全部public）
- 接口为解耦而生：“高内聚，低耦合”，方便单元测试
- 接口是一个“协约”，早已为工业生产所熟知（有分工必有协作，有协作必有协约）
- 它们都不能实例化，只能用来声明变量、引用具体类（concrete class）的实例

虽然说程序可以随便设计，但是最好还是遵循一下基本的设计原则，否则搞出一堆屎山到了后期就难以维护了。

抽象类是至少含有一个没有实现的抽象方法的类，需要用abstract修饰，与抽象类对应的就是具体类（平常用的就是）。注意abstract不能是private的，因为抽象方法要通过子类继承后实现；抽象方法不能带方法体；抽象类不能实例化；抽象方法在C++中叫纯虚函数。

```

abstract class Student() //抽象类（里面至少一个抽象方法，也可以包含不抽象的方法）
{
    abstract public void Study() ; //抽象方法（必须在抽象类里）
}

```

我们应该封装稳定不变、确定的成员，哪些不确定的有可能改变的成员就用抽象来修饰，后面的子类来实现。

下面的代码表示的就是经典的示例，不建议这么搞：

```

//错误示例
class Car
{
    public void Run()
    {
        Console.WriteLine("Car is Running");
    }
    public void Stop()
    {
        Console.WriteLine("Car is Stopped");
    }
}

```



```

    }
}
class Truck
{
    public void Run()
    {
        Console.WriteLine("Truck is Running"); //不建议Truck和Car一样的方法名
    }
    public void Stop()
    {
        Console.WriteLine("Turek is Stopped");
    }
}

```

此时可以新定义一个Vehicle类作为Car和Truck的父类，子类继承父类后改写对应的方法实现。

```

using System;
namespace Program
{
    class Program
    {
        static void Main(string[] args)
        {
            Vehicle car = new Car();
            car.Run();
            car.Stop();
            Vehicle truck = new Truck();
            truck.Run();
            truck.Stop(); //没有重写Stop将输出Vehicle的Stop方法
        }
    }
    //修改后
    abstract class Vehicle
    {
        abstract public void Run(); //抽象方法要用override重写
        public virtual void Stop() //虚拟方法也得用override重写
        {
            Console.WriteLine("Stopped");
        }
    }
    class Car:Vehicle //继承了抽象类，就必须实现抽象方法，否则自己得是抽象方法
    {
        public override void Run() //必须用override来实现抽象方法
        {
            Console.WriteLine("Car is Running");
        }
        public override void Stop() //重写父类方法
        {
            Console.WriteLine("Car is stopped");
        }
    }
    class Truck: Vehicle//继承了抽象类，就必须实现抽象方法，否则自己得是抽象方法
    {
        public override void Run()//必须用override来实现抽象方法
        {
            Console.WriteLine("Truck is Running");
        }
    }
}

```

```
//没有重写Stop将输出Vehicle的Stop方法
}
}
```

当然上面代码中Vehicle上面可以再订一个纯的抽象方法:

```
abstract class VehicleBase //纯抽象类
{
    abstract public void Stop();
    abstract public void Run();
}
```

这种纯抽象类其实就是接口interface了

```
interface VehicleBase //接口
{
    abstract public void Stop();
    abstract public void Run();
}
```

注意接口内方法必须是public的, 完整代码如下:

```
using System;
namespace Program
{
    class Program
    {
        static void Main(string[] args)
        {
            Vehicle car = new Car();
            car.Run();
            car.Stop();
            Vehicle truck = new Truck();
            truck.Run();
            truck.Stop(); //没有重写Stop将输出Vehicle的Stop方法
        }
    }

    interface IVehicle //接口 (一般是I开头)
    {
        abstract public void Stop();
        abstract public void Run();
    }

    //修改后
    abstract class Vehicle:IVehicle //继承接口
    {
        abstract public void Run(); //抽象方法要用override重写
        public virtual void Stop() //虚拟方法也得用override重写
        {
            Console.WriteLine("Stopped");
        }
    }

    class Car:Vehicle //继承了抽象类, 就必须实现抽象方法, 否则自己得是抽象方法
    {
        public override void Run() //必须用override来实现抽象方法
        {
            Console.WriteLine("Car is Running");
        }
        public override void Stop() //重写父类方法
        {

```

```

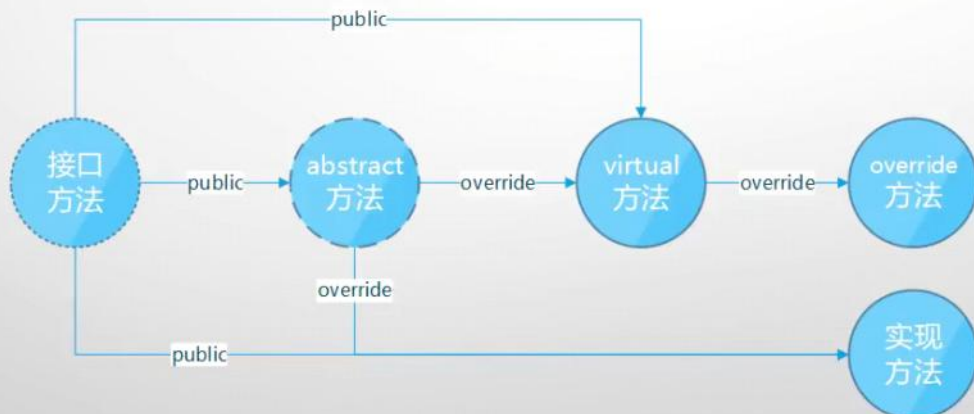
        Console.WriteLine("Car is stopped");
    }
}
class Truck: Vehicle//继承了抽象类，就必须实现抽象方法，否则自己得是抽象方法
{
    public override void Run()//必须用override来实现抽象方法
    {
        Console.WriteLine("Truck is Running");
    }
}
//没有重写Stop将输出Vehicle的Stop方法
}
}

```

6、接口

• 接口与单元测试

- 接口的产生：自底向上（重构），自顶向下（设计）
- C#中接口的实现（隐式，显式，多接口）
- 语言对面向对象设计的内建支持：依赖反转，接口隔离，开/闭原则



接口是供需双方都需要遵守的契约。

```

using System;
using System.Collections;
namespace Program
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] nums1 = new int[] { 1, 2, 3, 4, 5, };
            ArrayList nums2 = new ArrayList { 1, 2, 3, 4, 5 };
            Console.WriteLine(Sum(nums1)) ;
            Console.WriteLine(Avg(nums2));
        }
        static int Sum(int[] nums) //将一个数组传进来
        {
            int sum = 0;

```

```

        foreach (var n in nums) sum +=(int)n;
        return sum; //返回求和
    }
    static double Avg(ArrayList nums) //将一个数组链表传进来
    {
        int sum = 0; double count=0;
        foreach (var n in nums)
        {
            sum += (int)n;
            count++;
        }
        return sum / count;    //返回平均值
    }
}

```

上面代码没有使用接口，没有使用契约，可以发现这非常的不方便（两个方法的形参需要和实参的数据类型对应上），如果引入接口后，就可以改成这样：

```

using System;
using System.Collections;
namespace Program
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] nums1 = new int[] { 1, 2, 3, 4, 5, };    //ArrayList和数组都是遵守IEnumerable接口的
            契约的，因此可以直接传值
            ArrayList nums2 = new ArrayList { 1,2,3,4,5 };
            Console.WriteLine(Sum(nums1)) ;
            Console.WriteLine(Avg(nums2));
        }
        static int Sum(IEnumerable nums) //遵守IEnumerable接口的契约
        {
            int sum = 0;
            foreach (var n in nums) sum +=(int)n;
            return sum; //返回求和
        }
        static double Avg(IEnumerable nums) //遵守IEnumerable接口的契约
        {
            int sum = 0; double count=0;
            foreach (var n in nums)
            {
                sum += (int)n;
                count++;
            }
            return sum / count;    //返回平均值
        }
    }
}

```

接口的引入可以有效的降低程序的耦合度。耦合度高的代码最大问题是“牵一发而动全身”，而接口通过“面向接口编程”，遵循了“开闭原则”（对扩展开放、对修改关闭），这也是设计模式的核心思想。

比如做一个支付功能，最初只支持微信支付，如果直接依赖微信支付的具体类，后续要加支付宝支付，代码改动会很大，耦合度极高；但用接口封装

后就能彻底的解耦。

如以下接口实现的支付功能：

```
using System;
// 1. 定义支付接口：只规定“要做支付”这个行为，不关心具体怎么支付
public interface IPayment
{
    // 接口只定义方法签名，无实现
    bool Pay(decimal amount);
}
// 2. 微信支付实现类：实现接口的具体逻辑
public class WeChatPayment : IPayment
{
    public bool Pay(decimal amount)
    {
        Console.WriteLine($"微信支付{amount}元，扣减微信余额");
        return true;
    }
}
// 3. 支付宝支付实现类：新增实现类，完全不改动原有代码
public class AliPay : IPayment
{
    public bool Pay(decimal amount)
    {
        Console.WriteLine($"支付宝支付{amount}元，扣减支付宝余额");
        return true;
    }
}
// 4. 调用方（订单服务）：只依赖接口，不依赖具体支付类
public class OrderService
{
    // 接收接口类型参数，而非具体类
    public void ProcessPayment(IPayment payment, decimal amount)
    {
        // 调用方只关心“支付”这个行为，不关心是微信还是支付宝
        payment.Pay(amount);
    }
}
// 测试代码
class Program
{
    static void Main(string[] args)
    {
        OrderService orderService = new OrderService();
        // 用微信支付
        orderService.ProcessPayment(new WeChatPayment(), 100);
        // 改用支付宝支付：只改实例化的类，OrderService的代码一行不用动
        orderService.ProcessPayment(new AliPay(), 200);
        // 后续加银联支付：只新增一个实现IPayment的UnionPay类即可，完全不改动调用方
    }
}
```

没有接口时：如果 OrderService 直接依赖 WeChatPayment 类，要加支付宝支付，就得修改 OrderService 的代码（比如加 if-else 判断），调用方和实现方强耦合；

有接口时：OrderService 只依赖 IPayment 接口，新增任何支付方式（银联、银行卡），只需要新增一个实现 IPayment 的类，调用方代码完全不用改 —— 这就是接口把“调用逻辑”和“具体实现”拆分开，降低了耦合度。

在开发的时候我们应该尽可能的“松耦合”

接口隔离原则：不让类实现不需要的接口方法，比如做打印机系统，要是把打印、扫描、复印都塞到一个接口里，普通打印机就得被迫实现扫描 / 复印方法（要么空实现要么抛异常），既臃肿又容易出错。

```
// 臃肿的接口：包含打印机、扫描仪、复印机的所有功能
public interface IAllInOneMachine
{
    // 打印机功能
    void Print(string content);
    // 扫描仪功能
    void Scan(string filePath);
    // 复印机功能
    void Copy(string content);
}
// 普通打印机：只需要打印功能，但被迫实现所有接口方法
public class SimplePrinter : IAllInOneMachine
{
    public void Print(string content)
    {
        Console.WriteLine($"打印: {content}");
    }
    // 被迫实现不需要的方法，只能空实现/抛异常
    public void Scan(string filePath)
    {
        throw new NotSupportedException("普通打印机不支持扫描");
    }
    public void Copy(string content)
    {
        throw new NotSupportedException("普通打印机不支持复印");
    }
}
// 测试：调用不需要的方法会报错，代码臃肿且易出问题
class Program
{
    static void Main(string[] args)
    {
        IAllInOneMachine printer = new SimplePrinter();
        printer.Print("简历"); // 正常
        printer.Scan("test.jpg"); // 报错：不支持扫描
    }
}
```

但把接口拆成 IPrintable、IScannable、ICopyable 三个专用接口，普通打印机只实现 IPrintable，一体机实现全部，这样每个类只依赖自己需要的接口，代码更简洁、易维护，也从编译层面避免了调用不存在的功能。

```
// 拆分后的专用接口：每个接口只包含单一功能
public interface IPrintable
{
    void Print(string content); // 仅打印功能
}
public interface IScannable
{
    void Scan(string filePath); // 仅扫描功能
}
public interface ICopyable
{
}
```

```

        void Copy(string content); // 仅复印功能
    }
    // 普通打印机：只实现打印接口，无需关心其他功能
    public class SimplePrinter : IPrintable
    {
        public void Print(string content)
        {
            Console.WriteLine($"打印: {content}");
        }
    }
    // 多功能一体机：实现所有接口（因为它确实需要这些功能）
    public class AllInOnePrinter : IPrintable, IScannable, ICopyable
    {
        public void Print(string content)
        {
            Console.WriteLine($"打印: {content}");
        }
        public void Scan(string filePath)
        {
            Console.WriteLine($"扫描文件到: {filePath}");
        }
        public void Copy(string content)
        {
            Console.WriteLine($"复印: {content}");
        }
    }
    // 测试：客户端只调用自己需要的接口方法，无冗余
    class Program
    {
        static void Main(string[] args)
        {
            // 普通打印机：只暴露打印功能
            IPrintable simplePrinter = new SimplePrinter();
            simplePrinter.Print("简历"); // 正常
            // 编译报错：IPrintable接口没有Scan方法，从根源避免误用
            // simplePrinter.Scan("test.jpg");
            // 一体机：可调用所有功能
            AllInOnePrinter allInOne = new AllInOnePrinter();
            allInOne.Print("合同");
            allInOne.Scan("doc.pdf");
            allInOne.Copy("身份证");
        }
    }
}

```

但是不能玩的太过了，如果弄出一大堆单一接口，类接口的颗粒度就太小了，需要注意平衡。

7、反射和依赖注入

• 反射与依赖注入

- 反射：以不变应万变（更松的耦合）
- 反射与接口的结合
- 反射与特性的结合
- 依赖注入：此DI非彼DI，但没有彼DI就没有此DI..... ㄟ(ˋˋ)ㄟ

反射：程序在运行时，自己去查看一个类的结构（有哪些方法、字段），并直接调用它，而不是在写代码时就写死。反射就像照镜子，程序运行时通过名字就能发现类的结构、动态创建对象并调用方法。它能实现解耦，让我们通过配置（字符串）来控制调用具体哪个类，而不是写死代码。

```
// 1. 定义两个类
public class Hamburger { public void Make() { Console.WriteLine("做汉堡"); } }
public class ChickenWings { public void Make() { Console.WriteLine("做烤翅"); } }
// 2. 反射核心逻辑
public static void Cook(string dishName)
{
    // 不用 if-else 判断是谁
    // 直接根据字符串名字，去内存里“照镜子”找到对应的类
    var type = Type.GetType(dishName);
    var instance = Activator.CreateInstance(type); // 动态创建对象
// 动态调用方法
    type.GetMethod("Make").Invoke(instance, null);
}
// 3. 调用
Cook("Hamburger"); // 输出：做汉堡
Cook("ChickenWings"); // 输出：做烤翅
```

依赖注入：A类需要用到B类，B类不要自己造，而是由外部传进来。DI容器底层主要靠反射来实现，它通过配置或特性（Attribute）扫描程序集，动态创建对象并注入到需要的地方，从而实现了极低的耦合度。

假设有一个“电脑”类（Computer），它需要“显卡”（GPU）才能工作。

不使用DI是这样的：

```
public class Computer
{
    // 电脑自己去买显卡（强耦合）
    private NVidiaGPU gpu = new NVidiaGPU();
    public void ShowImage() { gpu.Render(); }
}
```

如果想换一张 AMD 的显卡，得拆开电脑（修改 Computer 类的代码），把 NVidia 拆下来，装上 AMD。使用DI是这样的：

```
public class Computer
{
    // 只规定需要显卡，但不关心是谁家的
    private IGPU gpu;
// 显卡由外部组装师傅传进来（注入）
    public Computer(IGPU gpu)
    {
        this.gpu = gpu;
    }
}
```

```
public void ShowImage() { gpu.Render(); }
}
```

// 调用方:

```
var myPC = new Computer(new AMDGPU()); // 传入AMD卡
var workPC = new Computer(new NVidiaGPU()); // 传入NVIDIA卡
```

此时电脑不用拆改, 想换什么显卡, 到手的时候递进去就行。

十七: 委托、Lambda、LINQ串讲

1、委托

委托是一种特殊的类, 引用类型的一种

不带参数和返回值的委托:

```
namespace Program
{
    class Fuck
    {
        static void Main(string[] args)
        {
            Mydele dele1 = new Mydele(M1); //dele1这个变量引用着一个MyDele类型的实例,
            实例里面放了一个M1方法
            dele1.Invoke(); //执行这个dele1实例 (输出1个M1方法)
            dele1 += dele1; //实例里面再重复放入一个dele1里面的方法
            dele1.Invoke(); //执行实例 (输出2个M1方法)
            dele1 += M2; //dele1里面再插入M2方法
            dele1.Invoke(); //输出2个M1方法和1个M2方法
            dele1(); //除了invoke还可以直接用调用方法那样直接调用, 因为委托的实例化就
            是方法
        }

        static void M1()
        {
            Console.WriteLine("M1 is called!");
        }
        static void M2()
        {
            Console.WriteLine("M2 is called!");
        }
        delegate void Mydele();
    }
}
```

带参数和返回值的委托:

```
namespace Program
{
    class Fuck
    {
        delegate int Mydele(int a, int b); //申明带参数的委托
        static void Main(string[] args)
        {
```

```

        Mydele dele = new Mydele(Fuck.Add); //委托插入格式一样的方法
        dele += Mul;
        int res = dele(100, 200); //执行实例返回里面方法的结果
        //先执行 Add(100,200) → 300
        //再执行 Mul(100,200) → 20000
        //最终输出 最后一个方法的结果: 20000
        Console.WriteLine(res);
    }
    static int Add(int x,int y)
    {
        return x + y;
    }
    static int Mul(int x,int y)
    {
        return x * y;
    }
}
}

```

泛型委托:

```

using System.Reflection.Metadata;
namespace Program
{
    class Fuck
    {
        delegate T MyDele<T>(T a , T b); //泛型委托(有时候没必要为了返回值不同而申明多个
        delegate类)
        static void Main(string[] args)
        {
            MyDele<int> deleAdd = new MyDele<int>(Add); //指明delegate是int
            int res = deleAdd(100, 200);
            Console.WriteLine(res);
            MyDele<double> deleMul = new MyDele<double>(Mul); //指明是double
            double mulRes = deleMul(100.32F, 200.12);
            Console.WriteLine(mulRes);
        }
        static int Add(int x,int y)
        {
            return x + y;
        }
        static double Mul(double x, double y)
        {
            return x * y;
        }
    }
}

```

常用泛型委托action: 对于没有返回值的函数, 我们用action来装他


```

using System.Reflection.Metadata;
namespace Program
{
    class Fuck
    {
        static void Main(string[] args)
        {
            Action action = new Action(M1);
            action();    //输出M1 is called
            Action<string> action1 = new Action<string>(SayHello);    //装有参数的方法可以用泛型委托
            action1("Jack");
            Action<string, string> action2 = new Action<string, string>(StringCom);    //装有2个参数的泛型委托
            action2("Jack", "Mary");
            Action<string, int> action3 = new Action<string, int>(SayHelloplus);    //装有不同类型的参数
            action3("牛逼", 3);
        }

        static void M1()
        {
            Console.WriteLine("M1 is called");
        }
        static void SayHello(string name)
        {
            Console.WriteLine($"Hello , {name}");
        }
        static void StringCom(string word1 , string word2)
        {
            Console.WriteLine(word1 + word2);
        }
        static void SayHelloplus(string name , int n)
        {
            for(int i= 0; i < n; i++)
            {
                Console.WriteLine($"Hello , {name}");
            }
        }
    }
}

```

常用泛型委托Func：对于有返回值的函数，我们用Function来装他

```

using System.Reflection.Metadata;
namespace Program
{
    class Fuck
    {
        static void Main(string[] args)

```

```

    {
        Func<int, int, int> func = new Func<int, int, int>(Add); //第一个参数是int,
        第二个参数是int, 返回值是int
        int res = func(100, 200);
        Console.WriteLine(res);
    }
    static int Add(int x, int y)
    {
        return x + y ;
    }
}

```

2、Lambda表达式

Lambda说白了就是匿名方法，还是inline内联的，很多小方法比如算个圆面积算个体积什么的，这种就没必要开一个不匿名的方法占空间，直接使用lambda表达式就行，需要知道lambda操作符=>，专门用来写短小的方法，配合委托、泛型、集合用

格式：(参数) => 要做的事

写法：

无参数 : () => Console.WriteLine("hello");
 一参数 : x => x * 10
 多参数 : (a,b) => a + b
 多行代码: (a,b) => { int sum = a + b; return sum * 2; };

配合委托：

```

using System.Reflection.Metadata;
namespace Program
{
    class Fuck
    {
        delegate int MyDele(int a, int b);
        static void Main()
        {
            // 左边：委托类型
            // 右边：Lambda 就是一个匿名方法！
            MyDele add = (a, b) => a + b;
            MyDele mul = (a, b) => a * b;
            Console.WriteLine(add(2, 3)); // 5
            Console.WriteLine(mul(2, 3)); // 6
        }
    }
}

```

配合泛型委托（可以不同自己定义delegate了）：

```

using System.Reflection.Metadata;

```

```

namespace Program
{
    class Fuck
    {
    static void Main()
        {
            // 定义：输入两个 int，返回 int
            Func<int, int, int> calc = (a, b) => a + b;
            int result = calc(10, 20); // 30
            // 输入 string，无返回
            Action<string> show = s => Console.WriteLine(s);
            show("hello lambda");
        }
    }
}

```

配合泛型集合：

```

using System.Reflection.Metadata;
namespace Program
{
    class Fuck
    {
    static void Main()
        {
            List<int> list = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            //筛选 (Where)
            // 选出 >5 的数字
            var big = list.Where(x => x > 5);
            // x 是列表里每一个元素
            //筛选+计算 (Where + Select)
            // 选出偶数，再乘以 2
            var evenDouble = list.Where(x => x % 2 == 0).Select(x => x * 2);
            //有一个满足就是true (Any)
            bool hasBig = list.Any(x => x > 100); // false
            //全部满足才能true (All)
            bool allBig = list.All(x => x > 0); // true
        }
    }
}

```

3、LINQ：.NET language Intergrated Query

LINQ = 给集合 / 数据用的 “超级查询语言”

可以让你像查数据库一样查 List、数组，代码超级简单、好读、好写。

例如需要找出>5的数字

```

namespace Program
{
    class Fuck
    {
    static void Main()
        {
            List<int> list = new List<int> { 1, 2, 3, 6, 7, 8 };
List<int> result = new List<int>();
            foreach (int num in list)
            {
                if (num > 5)
                {
                    result.Add(num);
                }
            }
        }
    }
}

```

如果使用LINQ:

```

namespace Program
{
    class Fuck
    {
    static void Main()
        {
            List<int> list = new List<int> { 1, 2, 3, 6, 7, 8 };
var result = list.Where(num => num > 5); //直接筛选
            foreach (int num in result)
            {
                Console.WriteLine(num);
            }
        }
    }
}

```

基本操作:

```

namespace Program
{
    class Fuck
    {
    static void Main()
        {
            List<int> list = new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
list.Where(x => x > 5);    // 找出所有大于5的数
list.OrderBy(x => x);      // 从小到大排序
list.Select(x => x * 2);   // 每个数 ×2
list.Any(x => x > 5);      // 有没有一个大于5?
        }
    }
}

```

```

        list.All(x => x > 5);    // 全部都大于5?
    }
}

```

十八、泛型

泛型就是泛化的数据类型，跟具体是相对的，类似于C++的模板+STL

泛型 = 「代码模板」，写代码时不把类型写死，用的时候再指定具体类型，一套代码适配所有类型。

在泛型出现前，想写一个「通用方法 / 类」，只能用 object 类型：

```

// 用 object 写的通用加法（有严重问题）
public static object Add(object a, object b)
{
    return (int)a + (int)b; // 硬转类型，风险极高
}

```

这种object类型性能差（装箱拆箱频繁）、安全性差（例如匹配个string类型就炸了）

泛型就是拿来解决这个问题的

场景	非泛型 (object)	泛型 (<T>)
性能	差（装箱拆箱）	高（无装箱拆箱）
类型安全	不安全（运行时报错）	安全（编译时检查）
代码复用	低（需要强转，逻辑复杂）	高（一套代码适配所有类型）
智能提示	无	有

```

using System.Reflection.Metadata;
namespace Program
{
    internal class Fuck
    {
        public static void Main(string[] args)
        {
            Apple apple = new Apple() { Color = "red" }; // 一个红苹果
            AppleBox box = new AppleBox() { Cargo = apple };
            Console.WriteLine(box.Cargo.Color);
            Book book = new Book() { name = "New Book" }; // 一本新书
            BookBox box2 = new BookBox() { Cargo = book };
            Console.WriteLine(box2.Cargo.name);
        }
    }

    class Book
    {
        public string name { get; set; }
    }

    class Apple // 苹果
    {
        public string Color { get; set; }
    }
}

```



```

    }
    class AppleBox    //箱子（装苹果）
    {
        public Apple Cargo { get; set; }
    }
    class BookBox    //装书箱子
    {
        public Book Cargo { get; set; }
    }
}

```

可以发现这段代码已经出现了“类型膨胀”的问题，两种产品需要两种对应的盒子，如果是1000种产品呢，那就非常麻烦了。解决方法可以定义一个Box类，再里面添加对应类型的成员。但是这样又会造成“成员膨胀”。这个时候就需要用泛型类了。

1、泛型类

泛型类就是类名后面加 <T>，整个类里都能用 T。

```

using System.Reflection.Metadata;
namespace Program
{
    internal class Fuck
    {
        public static void Main(string[] args)
        {
            Apple apple = new Apple() {Color = "Red" };
            Book book = new Book() { name = "New Book" };
            Box<Apple> box1 = new Box<Apple>() { Cargo = apple };
            Box<Book> box2 = new Box<Book>() { Cargo = book };
            Console.WriteLine(box1.Cargo.Color);
            Console.WriteLine(box2.Cargo.name);
        }
    }

    class Book
    {
        public string name { get; set; }
    }
    class Apple    //苹果
    {
        public string Color { get; set; }
    }
    class Box<TCargo>    //箱子（泛型类）
    {
        public TCargo Cargo { get; set; }
    }
}

```

当然还有类似于C++STL的系统定义好的泛型类：

- List<T> — 普通列表
- Dictionary<TKey, TValue> — 键值对快速查找
- HashSet<T> — 去重、快速判断存在
- Queue<T> — 排队（先进先出）

- `Stack<T>` — 栈（先进后出）

2、泛型接口

普通接口：类型写死，不通用，要重复写很多接口。

泛型接口：类型不写死，一套通用，安全、简洁、性能高。

```
using System.Reflection.Metadata;
namespace Program
{
    internal class Fuck
    {
        public static void Main(string[] args)
        {
            // 使用固定 int 的类
            MyClass c = new MyClass();
            c.Add(100);
            Console.WriteLine(c.Get()); // 输出100
            // 使用泛型类（可以任意指定类型）
            MyGenericClass<string> c2 = new MyGenericClass<string>();
            c2.Add("Hello");
            Console.WriteLine(c2.Get()); // 输出Hello
        }
    }

    // 泛型接口：接口名字后面 <T>
    interface IMyInterface<T>
    {
        // 方法参数、返回值都可以用 T
        void Add(T item);
        T Get();
    }

    // 类实现泛型接口，T → int
    class MyClass : IMyInterface<int>
    {
        private int _value;
        public void Add(int item)
        {
            _value = item;
        }
        public int Get()
        {
            return _value;
        }
    }

    // 泛型类 实现 泛型接口，T 继续传递
    class MyGenericClass<T> : IMyInterface<T>
    {
        private T _value;
        public void Add(T item)
        {
```

```

        _value = item;
    }
    public T Get()
    {
        return _value;
    }
}

```

3、泛型方法

就是方法自己带 <T>, 一个方法能处理任意类型。

```

using System.Reflection.Metadata;
namespace Program
{
    internal class Fuck
    {
        public static void Main(string[] args)
        {
            Print<int>(10);          // T = int
            Print<string>("你好"); // T = string
            Print<double>(3.14);    // T = double
        }
        // 泛型方法: 方法名后面 <T>
        static void Print<T>(T value)
        {
            Console.WriteLine(value);
        }
    }
}

```

4、泛型委托

最常用的泛型委托就是Action和Func, 这个不多说

以下说一下自定义的泛型委托

```

using System.Reflection.Metadata;
namespace Program
{
    internal class Fuck
    {
        public static void Main(string[] args)
        {
            MyDelegate<int, int> tmp = TMP;
            int res = tmp(200);
            Console.WriteLine(res);
        }
        static int TMP(int x)
        {
            return x;
        }
    }
}
// 自己定义泛型委托: 输入 T, 返回 TResult

```

```
        delegate TResult MyDelegate<T, TResult>(T value);  
    }  
}
```